

RESUMEN INGENIERÍA Y CALIDAD DE SOFTWARE – PRIMER PARCIAL

Ingeniería de Software

Software → conjunto de programas y documentación asociada necesaria para definir, desarrollar, operar y mantener un sistema. También puede entenderse como conocimiento representado en distintos niveles de abstracción, desde los requerimientos y modelos hasta el código ejecutable y el producto en producción. El mismo conocimiento acerca del sistema va tomando diferentes representaciones: requerimientos → prototipos → diseños → código → despliegue → sistema en producción.

Razones para no comparar SW y manufactura

- El SW es menos predecible
- No hay producción en masa, casi ningún producto de SW es igual a otro.
- No todas las fallas son errores.
- El SW no se gasta.
- El SW no está gobernado por las leyes de la física.

Ingeniería de Software → disciplina de ingeniería que aplica un enfoque sistemático, disciplinado y cuantificable a todos los aspectos de la producción de software, desde su concepción y desarrollo hasta su operación, evolución y mantenimiento. Surge porque construir software requiere mucho más que programar: involucra múltiples personas, actividades y versiones a lo largo de su vida.

Estado actual y antecedentes - Crisis del Software

Nacimiento formal del término Ingeniería de Software en 1968, en una conferencia de la OTAN. Después marca algunos hitos como *The Mythical Man-Month* de Brooks en 1975, *No Silver Bullet* en 1987, CMM/CMMI, el Manifiesto Ágil en 2001 y Lean Software Development en 2003.

El término Ingeniería de Software surge formalmente en 1968, en el contexto de la llamada Crisis del Software, frente a las dificultades crecientes para desarrollar sistemas cada vez más grandes y complejos de manera confiable, dentro del tiempo y costo previstos.

Crisis del Software → describe las dificultades recurrentes para construir software de calidad ante el crecimiento de la complejidad y demanda de los sistemas. Existen varios problemas típicos:

- el producto final no satisface las necesidades del cliente
- resulta difícil extenderlo o adaptarlo
- documentación deficiente
- mala calidad
- mayores tiempos y costos que los presupuestados

El hardware permitió construir sistemas cada vez mayores, pero la capacidad para desarrollar software no evolucionó al mismo ritmo. A esto se sumaron la creciente demanda, la subestimación de la complejidad, los cambios constantes y la falta inicial de una disciplina que abordara sistemáticamente todo el desarrollo.

La crisis no terminó completamente. Continúan existiendo errores de SW costosos y críticos. La diferencia es que hoy existen muchos más métodos, prácticas y herramientas para gestionarlos.

La crisis no puede resolverse únicamente mediante mejores tecnologías porque, según Brooks en *No Silver Bullet*, parte de la dificultad del desarrollo es esencial al propio software —complejidad, conformidad, mutabilidad e invisibilidad— y no puede eliminarse con una única herramienta o técnica.

Disciplinas que conforman la Ingeniería de Software

Disciplinas técnicas → las relacionadas directamente con la construcción del producto.

- Requerimientos
- Análisis y diseño

- Construcción
- Prueba
- Despliegue

Disciplinas de gestión → organizan y controlan el proyecto

- Planificación del Proyecto
- Monitoreo y Control

Disciplinas de soporte → atraviesan las demás actividades y ayudan a mantener la integridad y calidad.

- Gestión de Configuración de SW (SCM)
- Aseguramiento de calidad
- Métricas

Ejemplos de grandes proyectos de SW fallidos y exitosos

Fallidos:

- Mariner I (1962)
- Error de cálculo de Intel (1994)
- Mars Climate Orbiter (1999)

Exitosos:

- Microsoft
- Apple
- Google
- NVIDIA

Proceso de Software

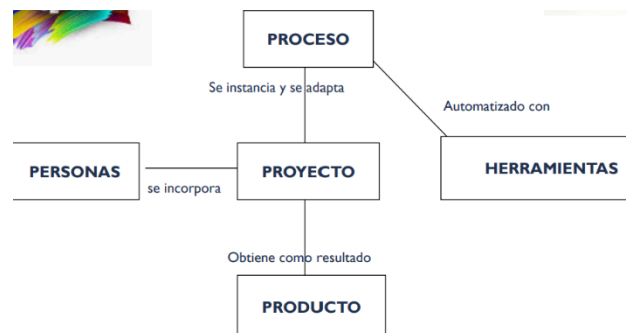
Conjunto estructurado de actividades utilizadas para desarrollar o mantener un sistema de SW. Las actividades concretas dependen de la organización y del tipo de producto que se construye. Además, para poder administrarlo, el proceso debe estar explícitamente modelado.

Otra definición → proceso de software es el conjunto de actividades, métodos, prácticas y transformaciones que las personas utilizan para desarrollar o mantener software y sus productos asociados.

Proceso vs Proyecto

Proceso → definición abstracta de cómo se debería desarrollar SW. Define actividades, métodos y prácticas.

Proyecto → instancia y adapta ese proceso a un caso concreto. Cuando comienza, se decide cuales de las actividades se aplicarán, como y con que personas y recursos.



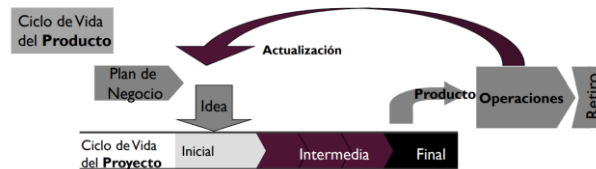
Elementos que intervienen en el proceso

- Procedimientos y métodos → definen que se hace y como.
- Personas → aportan habilidades, capacitación y motivación.
- Herramientas y equipos → apoyan y automatizan las actividades.

Ciclos de vida → serie de pasos a través de los cuales progresa un producto o un proyecto. Producto y proyecto tienen ciclos de vida diferentes.

Ciclo de vida del producto → comprende toda la existencia del producto, desde la idea inicial hasta que deja de utilizarse o es retirado. Puede incluir: Idea → desarrollo → producto → operación → actualizaciones → retiro. Por eso puede contener varios proyectos. Por ej. Uno crea la primera versión y años después otro desarrolla una actualización.

Ciclo de vida del proyecto → comprende desde el inicio hasta el cierre de un proyecto específico.



Ciclo de vida de un proyecto de software → es una representación del proceso desde una perspectiva determinada. Define principalmente:

- Que fases existen
- Que actividades ocurren
- En que orden se realizan

Por ej. Requerimientos → diseño → implementación → pruebas...

No son independientes del tipo de proyecto.

Relación proceso-ciclo de vida

El proceso define que actividades, métodos y practicas usamos, el ciclo de vida, como las distribuimos y ordenamos a lo largo del proyecto.

un mismo proceso de desarrollo puede organizarse mediante distintos ciclos de vida. Por eso, iterativo no significa necesariamente empírico, ni secuencial significa necesariamente definido. Los procesos definidos pueden utilizar distintos ciclos de vida; por ejemplo, PUD/RUP son procesos definidos, aunque utilicen un ciclo iterativo-incremental. En cambio, los procesos empíricos suelen requerir ciclos iterativos-incrementales, ya que necesitan inspección y adaptación frecuentes.

Procesos definidos y empíricos

Existen dos formas de controlar procesos.

- **Definidos** → intenta especificar de antemano cómo debe realizarse el trabajo y busca obtener resultados predecibles mediante la repetibilidad. La administración y el control se apoyan en la predictibilidad obtenida mediante la repetibilidad: se busca repetir el mismo proceso para obtener resultados similares. Ejemplos: PUD, RUP
 - Comienzan a fallar y de ahí surgen el agilismo, con los procesos empíricos.
- **Empíricos** → Ágiles (Scrum), Lean.

Hay situaciones donde puede ser favorable usar uno u otro.

Procesos empíricos → aquellos en los que las decisiones se toman principalmente a partir de la experiencia, la observación y los resultados obtenidos, en lugar de asumir que todo el proceso puede definirse y predecirse de antemano. Se utiliza especialmente cuando se trabaja con problemas complejos o con mucha incertidumbre, donde al comienzo no es posible conocer exactamente cuál será la mejor solución. Por eso se trabaja en ciclos cortos: se construye algo, se observa qué ocurrió, se aprende y se adapta el trabajo siguiente. De esta manera, el equipo va generando conocimiento a medida que avanza. El agilismo dice que la experiencia no es extrapolable, sino que se gana haciendo. Se basa en 3 pilares:

- **Transparencia** → el estado del producto y del trabajo debe ser visible y comprensible para poder evaluarlo correctamente.
- **Inspección** → se revisan frecuentemente el producto y el proceso para detectar problemas, desvíos u oportunidades de mejora.
- **Adaptación** → a partir de lo observado, se realizan los cambios necesarios en el producto, el plan o la forma de trabajar.

Estos tres pilares trabajando juntos, garantizan que el equipo vaya ganando experiencia.

Empírico no significa improvisado. Existe planificación, pero esa planificación se revisa y adapta continuamente según la evidencia obtenida.

Patrón de conocimiento en procesos empíricos

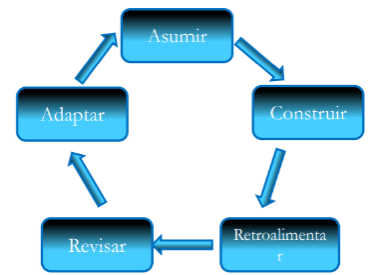
Inicialmente trabajamos sobre ciertas suposiciones; construimos algo, obtenemos evidencia de lo ocurrido, la analizamos y adaptamos nuestra forma de trabajar. Así generamos conocimiento a partir de la experiencia.

Retroalimentar ≠ Revisar

Retroalimentar → obtener feedback sobre lo que ocurrió.

Revisar → analizar e interpretar ese feedback para determinar que aprendimos.

A partir de ese aprendizaje, modificar el producto, proceso o decisiones.



Tipos de ciclos de vida

- **Secuencial** → las fases se ejecutan siguiendo un orden predeterminado. Una etapa conduce a la siguiente. Es adecuado cuando existe alta estabilidad y previsibilidad.
 - **Ventajas** → simple de planificar y seguir, las fases e hitos son claros y funciona bien cuando existe alta certeza y requerimientos estables.
 - **Desventajas** → poco flexible frente a cambios, requiere conocer gran parte de los requerimientos inicialmente y puede pasar mucho tiempo antes de obtener software funcionando. Un error u omisión detectado tarde puede ser costoso.
- **Iterativo** → El trabajo se realiza mediante iteraciones sucesivas. Cada iteración permite avanzar, aprender y normalmente obtener un entregable o incremento. Es el ciclo que se asocia principalmente a procesos empíricos y enfoques ágiles.
 - **Ventajas** → permite obtener entregas parciales y feedback temprano, incorporar cambios de requerimientos, reducir incertidumbre progresivamente y hacer visible el avance mediante incrementos.
 - **Desventajas** → necesita una arquitectura que soporte la evolución y una buena coordinación entre incrementos; si los cambios no se gestionan correctamente, la estructura del producto puede degradarse.
- **Recursivo** → El proceso vuelve sobre sí mismo, repitiendo determinadas etapas para refinar la solución.
 - **Ventajas** → permite volver sobre el problema y refinar progresivamente la solución, haciendo especial énfasis en identificar y reducir riesgos. Es útil para sistemas grandes y complejos.
 - **Desventajas** → la gestión puede ser más compleja y costosa, exige capacidad para identificar y analizar riesgos y puede resultar difícil establecer cuándo se ha avanzado lo suficiente para pasar al siguiente ciclo.

Los procesos definidos suelen asociarse con ciclos más predictivos, como el secuencial, mientras que los procesos empíricos requieren típicamente ciclos iterativos que permitan inspección y adaptación frecuente. No son conceptos equivalentes: proceso y ciclo de vida describen aspectos diferentes del desarrollo.

Criterios para seleccionar un ciclo de vida → No existe un ciclo de vida universalmente mejor. La elección depende de las características del producto, proyecto, equipo y contexto. Deben considerarse el nivel de incertidumbre y riesgo, estabilidad de los requerimientos, necesidad de entregas tempranas, tamaño y complejidad del producto, experiencia técnica del equipo, plazos disponibles y posibilidad de obtener feedback del cliente. La elección debe responder al contexto, no a una preferencia por una metodología.

- **Secuencial** → conviene cuando existe alta certeza, requerimientos claros y estables y no son necesarias entregas parciales.
- **Iterativo – incremental** → conviene cuando existe incertidumbre o volatilidad de requerimientos, se necesita feedback o el cliente requiere resultados parciales tempranos.
- **Recursivo** → conviene cuando el principal problema es gestionar riesgos elevados, especialmente en sistemas grandes o complejos.

Proyecto de Software

Esfuerzo temporal realizado para obtener un producto, servicio, o resultado único. Es la unidad concreta donde se adapta e instancia el proceso de desarrollo según las personas, recursos, objetivos y contexto disponibles. Permite adaptar el marco teórico del proceso, definiendo las tareas a realizar, las personas responsables y los mecanismos para gestionar riesgos, cambios y calidad.

Características:

- **Es único** → aunque dos proyectos busquen desarrollar productos similares, nunca son exactamente iguales: pueden cambiar las personas, recursos, fechas, etc. Todos los proyectos poseen características que los hacen únicos.
- **Tiene duración limitada** → son temporales: tienen principio y final. Termina cuando se alcanzan sus objetivos o cuando se decide cerrarlo. Una actividad repetitiva e indefinida, como una línea de producción, no es un proyecto.
- **Orientado a objetivos** → se realiza para conseguir ciertos objetivos, los cuales:
 - Deben ser claros y no ambiguos.
 - Deben poder evaluarse.
 - Tienen que ser alcanzables.
 - Sirven para determinar cuando el proyecto puede estar terminado.
- **Tiene tareas interrelacionadas, esfuerzos y recursos** → El proyecto no es simplemente una lista independiente de tareas. Las actividades tienen dependencias entre sí y consumen personas, tiempo, dinero y otros recursos. Esta interdependencia genera parte de la complejidad sistémica de gestionar un proyecto.

“La planificación lo es todo. Los planes no son nada” → el principal valor de la planificación no es generar un plan rígido, sino analizar anticipadamente objetivos, riesgos, recursos, estimaciones y alternativas. El plan puede necesitar modificaciones durante el proyecto.

¿Por qué es necesario gestionar el desarrollo de software? → porque el SW es complejo. A medida que pasamos de un programa aislado a un producto de software, aparecen más interfaces, integración, documentación, usuarios, cambios, restricciones y relaciones entre componentes. Por eso la gestión intenta mantener bajo control esa complejidad mediante planificación, coordinación, seguimiento y decisiones.

No Silver Bullet: esencia vs accidente → dos clases de dificultades al desarrollar SW:

Dificultades accidentales → problemas producidos por las herramientas o tecnologías utilizadas para construir software, pero que no pertenecen a la naturaleza misma del problema. Son problemas que la tecnología puede reducir o eliminar.

Dificultades esenciales → aquellas inherentes a construir software. No desaparecen simplemente porque aparezca una nueva tecnología. Se identifican cuatro:

- **Complejidad** → el SW esta compuesto por una gran cantidad de elementos diferentes y relaciones entre ellos. Al crecer el sistema, las interacciones aumentan y resulta cada vez más difícil comprenderlo y coordinarlo.
- **Conformidad** → El software debe adaptarse a reglas externas: necesidades del negocio, usuarios, leyes, sistemas preexistentes, interfaces, organizaciones, etc. Muchas de esas reglas pueden ser arbitrarias y el software igualmente debe respetarlas.
- **Mutabilidad** → El software está sometido continuamente a modificaciones porque cambian:
 - las necesidades de los usuarios
 - el negocio
 - la tecnología
 - las leyes
 - otros sistemas con los que interactúa.

- **Invisibilidad** → El software no posee una representación física natural que permita visualizar toda su estructura fácilmente. Esto hace más difícil comprender y comunicar sistemas grandes.

No Silver Bullet → Una bala de plata sería una tecnología o técnica capaz de solucionar de forma radical los problemas del desarrollo de software. La tesis de Brooks es que: No existe una única tecnología que pueda eliminar la complejidad esencial del desarrollo de software.

Nuevos lenguajes, IA, orientación a objetos o mejores herramientas pueden reducir dificultades accidentales, pero no eliminan la necesidad de comprender, especificar, diseñar y validar correctamente una solución compleja.

Implicancias de la complejidad del SW

- **Personas y tiempo no son intercambiables** → aumentar personas no reduce proporcionalmente la duración.
- **Ley de Brooks** → agregar personas a un proyecto atrasado puede atrasarlo más, por capacitación, coordinación y comunicación.
- **Costo de comunicación** → cuanto mayor es el equipo, mayor es la cantidad potencial de relaciones y esfuerzo de coordinación.
- **Integridad conceptual** → el diseño debe mantener una visión coherente, aunque participen muchas personas.
- **Efecto del segundo sistema** → evitar sobrecargar productos posteriores con funcionalidades y complejidad innecesarias.
- **Buena representación** → elegir estructuras y modelos adecuados puede simplificar la solución
- **Documentación útil** → ayuda a comunicar decisiones y mantener visibilidad, pero no elimina la complejidad esencial.

Ecosistema del desarrollo exitoso → el éxito del software no es un problema puramente tecnológico, sino también humano y arquitectónico. Requiere integridad conceptual para mantener una visión coherente del sistema, personas empoderadas capaces de colaborar y tomar decisiones, y un proceso orgánico y adaptable que permita aprender y responder al cambio. La combinación de estos elementos favorece la construcción de software resiliente y entregable en tiempo.

Administración de Proyectos

Busca, en términos simples, “lograr que el trabajo se haga” respetando tres condiciones principales: cumplir los objetivos/requerimientos, hacerlo en el tiempo acordado y dentro del presupuesto previsto. Aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer sus requerimientos.

Administrar un proyecto implica:

- Identificar los requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar especificaciones, planes y formas de trabajo a los distintos intereses de los stakeholders.

Triple restricción → tres variables que deben mantenerse equilibradas:

- **Objetivos** → que debe lograrse o entregarse.
- **Tiempo** → cuanto tiempo tenemos para entregarlo.
- **Costo** → cuanto puede gastarse.

El equilibrio entre estas variables afecta directamente la calidad del proyecto. Un proyecto de calidad entrega el resultado requerido, dentro del tiempo estipulado y respetando el presupuesto. Si cambia una variable, normalmente hay que ajustar alguna de las otras.

La responsabilidad del líder de proyecto es justamente balancear estas restricciones, que muchas veces compiten entre sí.

Rol del Líder de Proyecto → responsable de coordinar el proyecto, relacionarse con el equipo y los stakeholders y mantener el equilibrio entre alcance/objetivos, tiempo y costo. No se ocupa solamente de tareas técnicas. También debe coordinar personas, gestionar expectativas, comunicar decisiones y mantener alineados a los distintos involucrados. También participa en planificación, seguimiento, gestión de riesgos, control de cambios y comunicación del estado del proyecto.

Equipo de Proyecto → grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.

Características:

- Poseen conocimientos y habilidades diversos
- Pueden trabajar juntos eficazmente y generar sinergia → es importante ya que el resultado del equipo puede ser mejor que la suma de los aportes individuales, porque las habilidades se complementan.
- Suelen ser grupos relativamente pequeños
- Poseen un sentido de responsabilidad como unidad

Plan de Proyecto → un plan es a un proyecto lo que una hoja de ruta es a un viaje. Documenta las principales decisiones sobre cómo se ejecutará el proyecto. Integra las decisiones de alcance, proceso, calendario, recursos, riesgos, métricas y controles. Responde cuatro preguntas:

- ¿Qué hacemos? → alcance
- ¿Cuándo lo hacemos? → calendario
- ¿Cómo lo hacemos? → proceso, actividades y métodos
- ¿Quién lo hace? → responsables y asignación de trabajo

Planes de soporte → complementan al Plan de Proyecto para organizar actividades específicas, por ejemplo planes de testing, mantenimiento, calidad, subcontrataciones, capacitación y, si corresponde, planes de iteración.

¿Qué implica la planificación de un proyecto de software? → implica definir:

- Alcance del Proyecto
- Proceso y Ciclo de Vida
- Estimación
- Gestión de Riesgos
- Asignación de Recursos
- Programación de Proyectos
- Controles
- Métricas

Definición del alcance

Alcance del producto → conjunto de características y funcionalidades que debe poseer el producto o servicio. Responde a ¿Qué tiene que hacer el producto? Su cumplimiento se mide contra la ERS.

Alcance del proyecto → es todo el trabajo, y solo el trabajo, necesario para entregar ese producto con las características especificadas. Puede incluir, por ejemplo: relevamiento de requerimientos, diseño, programación, etc.

Responde a: ¿Qué trabajo tenemos que realizar para poder entregar el producto?

Su cumplimiento se mide contra el Plan de Proyecto

Definición de proceso y ciclo de vida → se selecciona y adapta el proceso y el ciclo de vida que se utilizarán, definiendo cómo y en qué orden se ejecutarán las actividades necesarias para alcanzar el objetivo del proyecto.

Influencia del ciclo de vida en la administración del proyecto → la elección del ciclo de vida condiciona la forma de planificar, monitorear y controlar el proyecto. En un ciclo secuencial, la gestión es más predictiva y se

apoya en una planificación inicial y en el cumplimiento de fases e hitos. En un ciclo iterativo, la planificación y el control se revisan frecuentemente a partir del feedback y del aprendizaje obtenido en cada iteración. En un ciclo recursivo, la administración se orienta especialmente a identificar y reducir riesgos en cada ciclo.

Estimaciones → En enfoques definidos/tradicionales existe una relación de derivación: Tamaño → Esfuerzo → Calendario → Costo → Recursos críticos. Por eso no deben estimarse estas variables de forma independiente.

Gestión de Riesgos

Riesgo → un evento que podría comprometer el éxito del proyecto. Mientras todavía no ocurrió, es un riesgo.

Se analiza principalmente mediante dos variables:

- Probabilidad: posibilidad de que ocurra.
- Impacto: gravedad de las consecuencias si ocurre.

Proceso → Identificar → Analizar → Planificar → Seguimiento y Control → Aprendizaje (ciclo)

- Identificación → Detectar posibles eventos que puedan afectar el proyecto y construir una lista de riesgos.
- Análisis → Evaluar cada riesgo según su probabilidad e impacto. Priorizar.
- Planificación → Definir qué se hará frente al riesgo.
- Seguimiento y control → Los riesgos no se analizan una sola vez. Deben revisarse durante todo el proyecto. La lista se actualiza continuamente y un mismo riesgo puede tener distinto impacto en momentos diferentes
- Aprendizaje → La experiencia obtenida alimenta una base de conocimiento de riesgos, que puede ayudar a reconocer situaciones similares en el futuro.

Riesgos típicos → dependencia de proveedores externos, mala identificación de las necesidades del cliente, etc.

Métricas de Software → permite obtener información cuantitativa acerca del estado o comportamiento del software, proyecto o proceso. Se dividen en tres dominios:

- **Métricas de proceso** → Miden el comportamiento del proceso de desarrollo a nivel más general u organizacional. Se usan principalmente para analizar y mejorar cómo trabaja la organización.
- **Métricas de proyecto** → Sirven para detectar desvíos y tomar acciones correctivas. Miden aspectos de un proyecto particular, como:
 - Esfuerzo
 - Calendario
 - Costos
 - cumplimiento del plan.
- **Métricas de producto** → Miden características del software construido, por ejemplo: tamaño, defectos, otras características de calidad.

Las métricas de distintos proyectos pueden consolidarse para generar métricas de proceso a nivel organizacional.

Métricas básicas de un proyecto de SW

Tamaño del producto y defectos → producto

Esfuerzo y calendario → proyecto

- Una métrica debe responder a una necesidad de información concreta. No todas las métricas son útiles para todos los roles.
- Las métricas tienen que ser útiles y proporcionales a la necesidad. Medir más o con mayor precisión no necesariamente mejora la gestión; solo deben utilizarse métricas que aporten información práctica para tomar decisiones.
- La función de las métricas es justamente hacer visible cómo se está comportando el proyecto respecto de esas restricciones, para poder decidir si es necesario actuar.

Monitoreo y control

Monitoreo → observar y medir periódicamente qué está ocurriendo realmente en el proyecto. Se utilizan las métricas.

Control → comparar esa situación real con lo que estaba planificado y tomar acciones si existen desviaciones.

Tres factores principales para el éxito

- **Monitoreo y feedback** → permite detectar tempranamente problemas y desviaciones y aprender de lo que realmente ocurre.
- **Misión u objetivo claro** → todos deben entender qué intenta lograr el proyecto; de lo contrario, pueden trabajar eficientemente, pero en direcciones distintas.
- **Comunicación** → permite coordinar personas, compartir información, resolver problemas y mantener alineados al equipo y los stakeholders.

Causas de fracaso de proyectos

- fallas al definir el problema;
- planificación basada en datos insuficientes;
- planificación realizada sin participación adecuada de quienes ejecutarán el trabajo;
- falta de seguimiento del plan;
- plan pobre en detalles;
- mala planificación de recursos;
- estimaciones basadas solamente en supuestos y no en evidencia histórica;
- ausencia de una persona claramente responsable.

Muchos proyectos no fracasan por un problema puramente técnico, sino por deficiencias en objetivos, planificación, información, comunicación, seguimiento y responsabilidades.

Vínculo proceso–proyecto–producto (4P): el proceso proporciona el marco general de actividades, métodos y prácticas para desarrollar software. Cada proyecto selecciona y adapta ese proceso según sus objetivos, personas, recursos, riesgos y características particulares. Las personas ejecutan esas actividades, apoyadas por herramientas, y como resultado del proyecto se obtiene un producto de software. Por lo tanto: el proceso se adapta e instancia en un proyecto, y el proyecto produce el producto. Las características del producto condicionan el proceso elegido, y el proyecto adapta ese proceso al contexto concreto para obtener dicho producto.

Fundamentos ágiles

Corriente ágil → adhiere al empirismo. Se basa en 4 valores y 12 principios.

Valores del agilismo

- **Individuos e interacciones sobre procesos y herramientas** → Lo más importante son las personas y cómo se comunican y colaboran. Los procesos y herramientas ayudan, pero no reemplazan una buena interacción entre quienes participan del proyecto.
- **Software funcionando sobre documentación extensiva** → La documentación sigue siendo necesaria, pero el objetivo principal es entregar software que funcione y aporte valor. En lugar de invertir demasiado tiempo en documentación exhaustiva desde el comienzo, se busca producir incrementos utilizables y documentar lo necesario.
- **Colaboración con el cliente sobre negociación contractual** → En vez de definir todo al principio y limitarse a cumplir un contrato, se busca una relación continua con el cliente para comprender mejor sus necesidades, recibir feedback y ajustar el producto. Esto es importante porque los cambios son constantes y muchas veces el usuario termina precisando lo que realmente necesita al ver el producto.
- **Respuesta al cambio sobre seguir un plan** → El plan sirve como orientación, pero no debe impedir adaptarse cuando aparecen nuevos requisitos, necesidades o información.

Principios del Manifiesto Ágil

#	Principio	Explicación breve	Valor más relacionado
1	La mayor prioridad es satisfacer al cliente	La prioridad es entregar valor al cliente de manera temprana y continua, sin esperar al final del proyecto.	Colaboración con el cliente / Software funcionando
2	Aceptar cambios en los requisitos	Los cambios son esperables, incluso en etapas avanzadas, y deben aprovecharse para mejorar el producto.	Respuesta al cambio
3	Entregar software funcional frecuentemente	Trabajar en ciclos cortos y producir incrementos utilizables permite obtener valor y feedback rápidamente.	Software funcionando
4	Negocio y desarrolladores trabajan juntos	Técnicos y personas del negocio deben colaborar continuamente para mantener una comprensión común de las necesidades.	Colaboración con el cliente / Individuos e interacciones
5	Trabajar con individuos motivados	Se debe dar al equipo el entorno, apoyo y confianza necesarios para que pueda realizar el trabajo.	Individuos e interacciones
6	Comunicación cara a cara es más eficiente	La conversación directa es la forma más eficiente de transmitir información y resolver dudas rápidamente.	Individuos e interacciones
7	El software funcionando mide el avance	El progreso real no se mide principalmente por documentos o tareas realizadas, sino por funcionalidad que efectivamente funciona.	Software funcionando
8	Desarrollo sostenible	El equipo debe mantener un ritmo de trabajo que pueda sostener en el tiempo, evitando esfuerzos extremos permanentes.	Individuos e interacciones
9	Excelencia técnica y buen diseño	Mantener calidad técnica y buen diseño facilita modificar, mantener y evolucionar el producto.	Software funcionando
10	La simplicidad es esencial	Se busca hacer solo el trabajo necesario y evitar funcionalidades, procesos o documentación que no aporten valor.	Software funcionando / Respuesta al cambio
11	Equipos autoorganizados	El propio equipo decide cómo organizar el trabajo; de estos equipos surgen mejores requisitos, diseños y arquitecturas.	Individuos e interacciones
12	Reflexionar y adaptarse periódicamente	El equipo revisa regularmente cómo está trabajando y modifica su comportamiento para mejorar.	Respuesta al cambio

¿Qué es ágil? → NO es una metodología o proceso. Es una ideología con un conjunto definido de principios que guían el desarrollo del producto. El objetivo principal es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se usan en entornos con una gran variabilidad de requerimientos,

involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación. Valores de los equipos ágiles:

- Planificación continua, multi-nivel
- Facultados, auto-organizados, equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing Concurrente

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema.

Ágil entonces es un balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación, sin embargo, eso no es lo más importante:

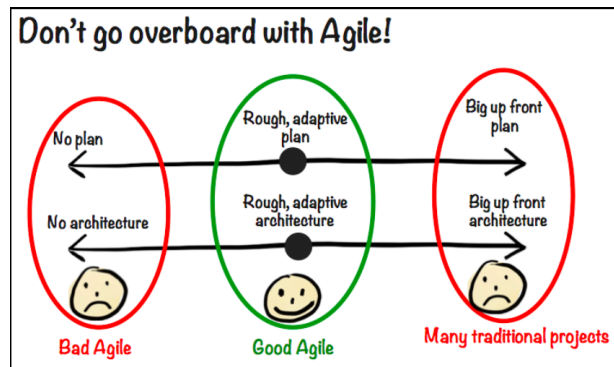
- Los métodos ágiles son adaptables en lugar de predictivos.
- Los métodos ágiles son orientados a la gente en lugar de orientados al proceso.

La filosofía ágil se materializa en el **manifiesto ágil**. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos.

¿Cuándo es aplicable ágil?

Los frameworks ágiles son distintos marcos de trabajo para implementar los principios y valores ágiles y generalmente son orientados a la gestión de proyectos y no al desarrollo. Las metodologías ágiles son aplicables en el contexto de lo complejo (dan buenos resultados), en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial.

Ser ágil no es ser indisciplinado



¿Por qué ir a ágil?

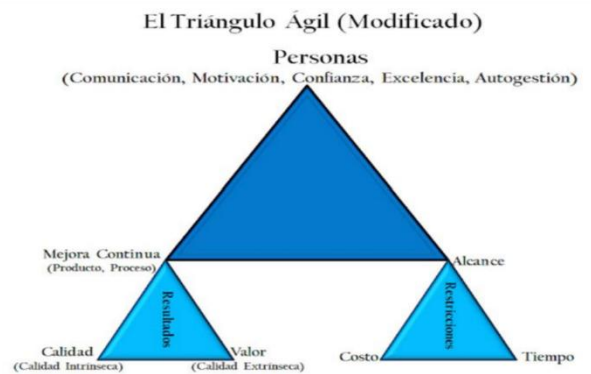
- Acelera entregas de productos de valor para el cliente.
- Permite adaptarse fácilmente a cambios de prioridades y requerimientos.
- Permite alinear los objetivos organizacionales y de TI.
- Reduce riesgos de proyecto.
- Permite una mejor visibilidad del proyecto.
- Mejora las relaciones del equipo.
- Reduce costos del proyecto.

El triángulo ágil

Apunta a que lo que realmente tiene valor es el software funcionando para el cliente, por lo que se centra más en la calidad del producto, es decir, en la calidad intrínseca (las características de calidad que nosotros le ponemos al producto para que el producto pueda satisfacer al cliente) y la calidad extrínseca (el valor que el producto tiene para el cliente).

El triángulo ágil (modificado) apunta a un producto de software que pone a las personas (que tienen ciertas características como confianza, motivación, etc.) en el lugar más importante ya que las personas son las que hacen el software. La intención de mejora continua que tienen las personas en el proyecto del producto y del proceso.

- Las personas son lo más importante. Estas son las que construyen el software.
- Debajo tenemos el pie, en el cual está la mejora continua (tanto del producto con las iteraciones como del proceso, cuando habla el equipo entre sí). Es el triángulo de resultados, teniendo como pie a la calidad (la cual es calidad intrínseca, ésta representa a las características que las personas le ponen al producto para que pueda satisfacer al cliente) y al valor (calidad extrínseca, que es el valor que el producto tiene para el cliente).
- Del otro lado de la base podemos encontrar el alcance, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.

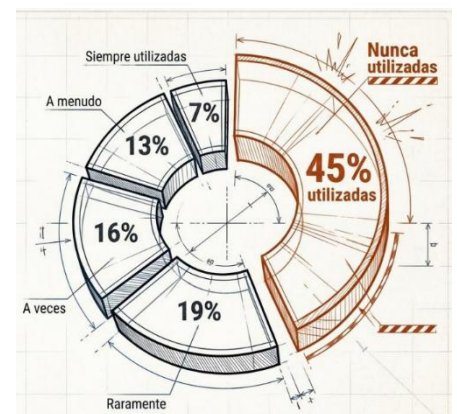


Gestión de Productos

Busca decidir que producto conviene construir y cómo hacerlo evolucionar para generar valor real para los usuarios y el negocio. El foco no está solamente en desarrollar correctamente funcionalidades, sino en asegurarse de que esas funcionalidades respondan a necesidades reales y sean en efecto utilizadas.

El mayor riesgo a la hora de construir SW no es fallar técnicamente, es construir a la perfección algo que nadie quiere.

Valor vs. Desperdicio → Producir más SW o desarrollar más rápido no implica necesariamente entregar más valor. Según la mirada Lean, una característica genera valor cuando produce un beneficio real para el cliente o usuario. Si una funcionalidad se construye, pero no responde a una necesidad, el esfuerzo destinado a la misma es desperdicio. Como solo una pequeña proporción de las funcionalidades se utiliza siempre, el objetivo no debe ser maximizar la cantidad de features, sino descubrir cuáles son realmente importantes para el usuario.



Del producto funcional a la experiencia de usuario

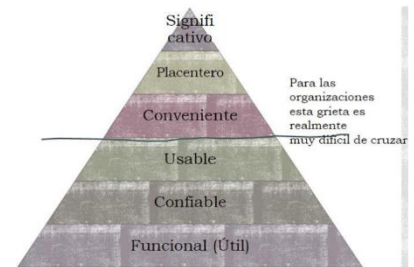
Hay un cambio de enfoque → no alcanza solamente con pensar en las tareas y características técnicas del producto. Hay que pensar también en la experiencia que tendrá la persona al usarlo. Ya no solo alcanza con satisfacer la necesidad, hay que pensar en la experiencia.

Un producto evoluciona aproximadamente desde características básicas hacia una experiencia cada vez más valiosa: **Funcional** → **Confiable** → **Usable** → **Conveniente** → **Placentero** → **Significativo**

La base sigue siendo necesaria, si un SW no funciona o no es confiable, difícilmente genere valor. Pero en mercados donde existen varias soluciones que cumplen la misma función, eso puede no ser suficiente. La

diferenciación empieza a aparecer en qué tan fácil, agradable y significativa resulta la experiencia para el usuario.

- Funcional: hace lo que tiene que hacer.
- Confiable: el usuario puede depender de que funcione correctamente.
- Usable: permite alcanzar los objetivos de manera eficiente y comprensible.
- Conveniente: facilita realmente la vida o el trabajo del usuario.
- Placentero: genera una experiencia agradable.
- Significativo: llega a producir un impacto importante en la forma en que el usuario realiza una actividad.



Brecha del ingeniero → la Gestión del Producto requiere pasar de pensar exclusivamente en tareas, funcionalidades, y tecnología a comprender también a las personas, su contexto y su experiencia. El objetivo no es solo construir correctamente el SW, sino construir el producto correcto para sus usuarios.

Ciclo Lean Startup

Ya habiendo dicho que no sabemos con certeza que funcionalidades aportaran valor, aparece el problema:

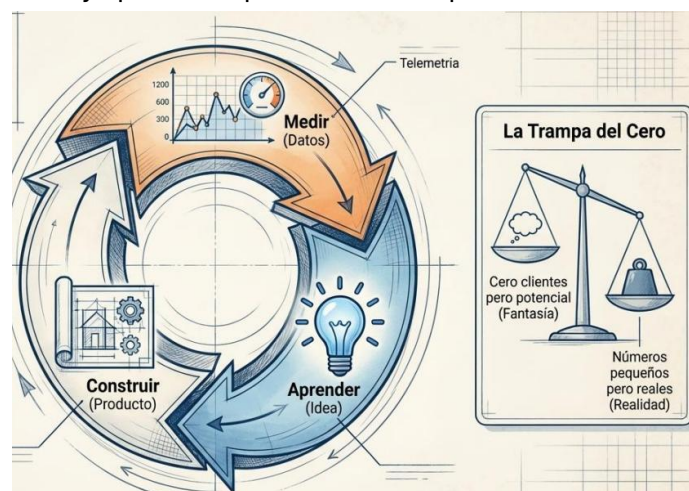
¿Cómo descubrimos que quiere realmente el usuario sin construir todo el producto primero?

Lean Startup propone trabajar mediante un ciclo de aprendizaje continuo: Construir → Medir → Actualizar

1. **Construir** → se crea una versión, experimento o representación del producto que permita poner a prueba una idea.
2. **Medir** → Se observa cómo reaccionan usuarios reales y se recolectan datos.
3. **Aprender** → con esos resultados se determina si las hipótesis iniciales eran correctas o si deben modificarse.

Después, ese aprendizaje alimenta una nueva construcción y el ciclo vuelve a comenzar. Se construye algo para evaluarlo, modificar lo planteado inicialmente, y volver a construir. Su objetivo es reducir la incertidumbre y evitar invertir grandes cantidades de esfuerzo en un producto que el mercado podría no querer.

¿Por qué trabajar así? → porque esperar hasta estar completamente seguros antes de lanzar algo genera la llamada “trampa del cero”: cero usuarios, cero feedback y, por lo tanto, muy poco aprendizaje real. En vez de intentar eliminar toda incertidumbre antes de construir, Lean Startup propone poner cuanto antes una hipótesis frente a usuarios reales y aprender a partir de su comportamiento.



Producto Mínimo Viable (MVP)

Versión mínima de un producto, que permite poner a prueba una hipótesis con usuarios reales y obtener aprendizaje validado con el menor esfuerzo posible. Sirve para los early adopters.

La finalidad principal del MVP no es vender ni construir el producto definitivo, sino aprender si la idea

realmente genera interés y valor antes de invertir demasiado tiempo y dinero. Busca obtener feedback del cliente invirtiendo el mínimo esfuerzo hasta saber si existe interés genuino.

Siguiendo con el ciclo Lean Startup, un MVP funciona como un experimento para confrontar la realidad lo antes posible, no como una versión final. Requiere simplificación y evitar la sobre-construcción.

- La audacia de cero → 0 ingresos – 0 clientes. El número cero invita a la pura imaginación de los inversores. Esto crea un incentivo brutal para retrasar el lanzamiento hasta que el producto sea “perfecto”.
- El costo del retraso → posponer el MVP aumenta el esfuerzo desperdiciado, pierde feedback temprano esencial y maximiza el riesgo de construir algo que nadie quiere.

Hay que cuidar los términos de “mínimo” y “viable”:

- **Mínimo** → construir solo lo estrictamente necesario para aprender.
- **Viable** → lo que se presenta debe ser suficientemente coherente como para que el usuario pueda evaluarlo de manera válida.

No sirve reducir tanto el producto que el usuario tenga una mala experiencia simplemente porque está incompleto. Enfocarse solamente en “mínimo” y olvidarse de “viable” impide obtener feedback útil.

El MVP debe construirse como una “feta vertical” → debe contener un poco de cada capa necesaria para permitir que el usuario experimente una funcionalidad de principio a fin: interfaz + lógica + datos + comportamiento real. El MVP tiene que ser cohesivo, aunque limitado.

Hipótesis de valor e hipótesis de crecimiento

El MVP sirve para validar dos grandes tipos de hipótesis.

Hipótesis de valor → ¿Este producto realmente aporta valor al usuario? Se observa si las personas lo usan vuelven a utilizarlo, lo encuentran útil o resuelve efectivamente su problema. Si no se valida, puede aumentar el riesgo de desperdicio, construir algo que nadie quiere y de sobre-ingeniería. Una posible métrica es la tasa de retención → si los usuarios regresan al producto, hay una señal de que están encontrando valor.

Hipótesis de crecimiento → ¿Cómo llegan nuevos usuarios al producto y puede crecer su adopción? Ya no se pregunta solamente si el usuario actual obtiene valor, sino si el producto tiene la capacidad de atraer a otros.

No siempre hace falta construir todo

Un MVP no necesariamente tiene que ser un sistema completo, ni siquiera código funcionando en sentido tradicional. Pueden usarse formas de validación más baratas.

Prueba de humo → permite verificar interés antes de construir el producto real. Por ejemplo, se puede mostrar una landing page, un video o una propuesta simulada y medir cuántas personas muestran interés.

Prototipo → ya representa parcialmente cómo sería el producto, pero todavía puede tener funcionalidades incompletas o simuladas. Sirve para observar como interactúa el usuario; si entiende el flujo; si la solución propuesta resuelve el problema; que aspectos deberían modificarse.

UVP (Unique Value Proposition) – Propuesta de Valor Única

Es la hipótesis principal acerca de que hace que el producto sea valioso y diferente para sus usuarios. Tengo una propuesta de valor única como hipótesis inicial y hago el MVP para probar esa hipótesis.

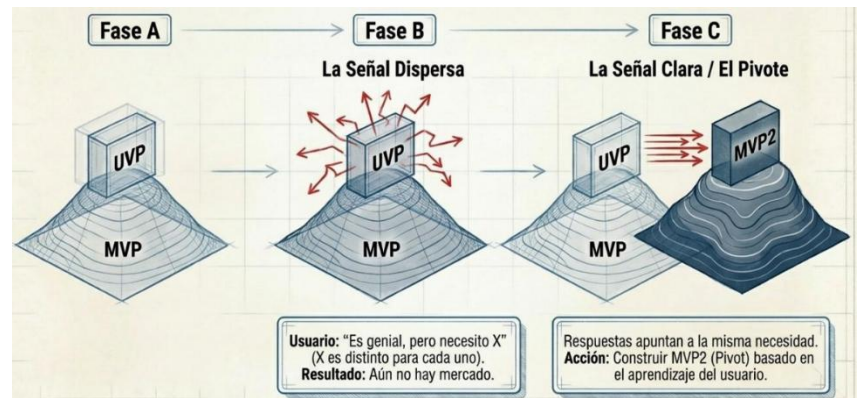
UVP → lo que creo que generará valor.

MVP → el experimento con el que pruebo si esa creencia es cierta.



Pivot: qué hacemos si la hipótesis no se confirma

El MVP también es una hipótesis. Puede ocurrir que lo mostremos a distintos usuarios y recibamos respuestas muy diferentes; por ejemplo, cada uno dice que lo usaría “si tuviera X”, pero ese X cambia para cada persona. Eso indica que todavía no encontramos una necesidad común suficientemente clara.



En cambio, si muchas respuestas empiezan a apuntar hacia la misma necesidad, aparece una señal más fuerte. En ese caso se puede pivotar: modificar la hipótesis inicial de cliente, problema o solución utilizando lo aprendido y construir un nuevo MVP orientado a esa nueva dirección. No debería ser un problema tener que construir varios MVP hasta validar la hipótesis. Iterar → mejorar o ajustar manteniendo esencialmente la misma hipótesis. Pivotar → cambiar una hipótesis importante porque la evidencia muestra otra dirección.

¿Qué debe entrar en el MVP?

Se debe armar una **matriz de priorización**. El MVP contiene únicamente lo necesario para poner a prueba la propuesta de valor. Todo aquello que no contribuya a validar esa hipótesis debe postergarse, delegarse o eliminarse.



Escalando el producto

Una vez validamos el MVP, aparecen dos situaciones distintas:

1. Hay áreas donde tenemos alta certeza de que existe valor.
2. Hay áreas que parecen prometedoras, pero todavía tienen alta incertidumbre.

MMF (Minimum Marketable Feature) – Característica Mínima Comercializable → porción mas pequeña de funcionalidad que puede liberarse aportando valor comercial inmediato. Se usa cuando existe alta certeza de valor: ya tenemos evidencia suficiente de que esa área resulta valiosa y queremos entregar la mínima funcionalidad que permita aprovecharla.

Sus objetivos son:

- Aportar valor rápidamente
- Reducir el Time to Market
- Evitar construir una funcionalidad enorme antes de obtener beneficios
- Permitir seguir incorporando MMF sucesivas

MVF (Minimum Viable Feature) – Característica Mínima Viable → aparece cuando identificamos un área potencialmente valiosa, pero existe alta incertidumbre acerca de si una determinada feature funcionará realmente. Es como llevar la lógica del MVP al nivel de una característica particular. Se implementa la versión mínima de esa funcionalidad que permita utilizarla realmente y obtener aprendizaje de usuarios.

Si funciona, luego podemos desarrollar nuevas MMF en esa área. Si no funciona, podemos cambiar el enfoque antes de invertir demasiado. La presentación la describe como una función “pionera” utilizada para generar aprendizaje continuo.

Diferencias:

MMF	MVF
Alta certeza de valor	Alta incertidumbre
Busca entregar valor comercial	Busca generar aprendizaje
Sabemos que queremos desarrollar esa área	Todavía estamos comprobando si conviene
Optimiza Time to Market	Reduce riesgo de construir algo incorrecto

Relación con requerimientos ágiles → Desde el punto de vista del negocio, un MVP, MMF o MVF ya representa la unidad mínima que tiene sentido construir para obtener aprendizaje o valor. Pero desde el punto de vista del equipo de desarrollo, esa unidad todavía puede ser demasiado grande para implementarla directamente. Por eso se divide en User Stories, que actúan como cortes pequeños de ejecución técnica. Las US permiten:

- reducir riesgo tecnológico
- implementar gradualmente
- recibir feedback técnico y funcional temprano
- evitar intentar desarrollar un bloque enorme de una sola vez.

Ecosistema de crecimiento del producto → una vez logrado cierto encaje producto – mercado, el producto puede evolucionar combinando MVFs (para explorar nuevas áreas de valor) y MMFs (para explotar certezas).



La lógica es: descubro mediante MVP → crezco mediante MMF → sigo explorando nuevas oportunidades mediante MVF.

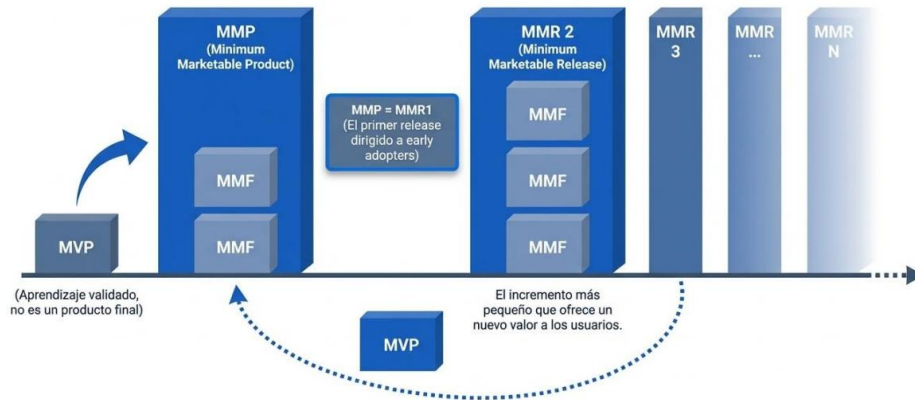
Pasar al producto comercializable

MMP (Minimum Marketable Product) – Producto Mínimo Comercializable → producto mínimo que ya tiene suficiente funcionalidad y valor como para salir efectivamente al mercado.

MMR (Minimum Marketable Release) → incremento mínimo del producto que puede liberarse aportando nuevo valor a los usuarios.

La secuencia sería: **MVP → MMP / MMR1 → MMR2 → MMR3 → ...**

El primer reléase comercializable puede considerarse: MMP = MMR1 y luego cada MMR sucesivo agrega nuevas características de valor.



Roadmap del producto → todas las decisiones se representan en uno. Muestra a alto nivel como se espera que evolucione el producto a través del tiempo.



MLP (Minimum Lovable Product) – Producto Mínimo Adorable → producto mínimo que, además de ser viable y útil, busca generar una experiencia positiva y diferenciadora que produzca conexión emocional, lealtad y deseo de continuar utilizando el producto. La idea aparece porque, en mercados maduros y competitivos, un producto puede ser técnicamente correcto y aun así no diferenciarse. El foco ahora es: ¿Por qué el usuario elegiría este producto y querría seguir usándolo? Lo viable ya no es suficiente; hay que hacer algo memorable.

Hay 3 razones principales para llegar al MLP:

- **Reputación de marca** → demostrar a los usuarios que te importan sus necesidades.
- **Diferenciación** → cuando la funcionalidad básica ya es un estándar, hay que destacar de otra manera.
- **Feedback superior** → una experiencia positiva genera usuarios más comprometidos y una retroalimentación más rica.

La adorabilidad no depende solo de agregar funciones, sino de reducir fricciones y generar una experiencia de uso que produzca satisfacción, confianza, motivación y vínculo con el producto.

Comparación MVP vs MMP vs MLP

	MVP (Viable)	MMP (Comercializable)	MLP (Adorable)
Objetivo Principal	Validar concepto de forma económica	Atender a base amplia con funcionalidad clave	Crear conexión emocional e interacción profunda
Enfoque de Diseño	Investigar y empatizar	Construir y refinar / Código y testing	Diseñar para el deleite / Impacto emocional
Beneficio Principal	Informar el alcance del producto	Asegurar la calidad y salida al mercado	Aumentar la lealtad y el amor del usuario
Ejemplo Típico	Entrevistas y videos	Lanzamiento Beta	App con animaciones UI divertidas y personalizadas

Diferencias:

MVP se enfoca en aprendizaje.
MMF aporta valor comercial concreto.
MLP prioriza deleite emocional.

Estrategia a lo largo del ciclo de vida – Resumen

Etapas MVP → objetivo: maximizar aprendizaje y reducir incertidumbres. Se experimenta, se valida y se puede pivotar.

Etapas MMP / MMF → Cuando ya hay evidencia de valor, el foco pasa a: Entregar valor comercial y escalar la tracción. Se agregan MMF en áreas conocidas y MVF cuando todavía existen hipótesis por probar.

Etapas MLP → Cuando el producto ya es competitivo y maduro, el foco pasa a elevar la experiencia, diferenciarse y evitar que los usuarios abandonen el producto.

El **ciclo se reinicia** continuamente: nuevas hipótesis, nuevos MVF y nuevos pivotes.

Requerimientos ágiles

El fondo de la cuestión está en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. Esto último implica una predisposición, una actitud del equipo frente a la posibilidad de cambios. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido.

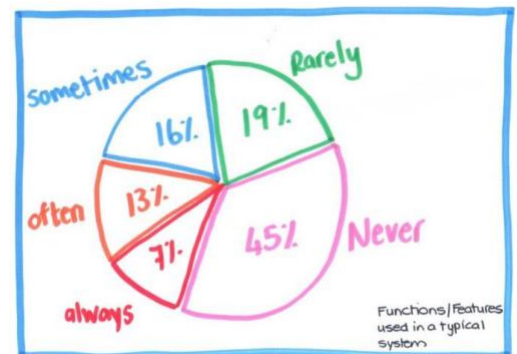
Los requerimientos ágiles se basan en tres pilares:

- Usan el valor para construir el producto correcto → Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio.
- Determinar que es solo lo suficiente → los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Este pilar postula que se tiene que empezar con lo mínimo necesario para generar realimentación y alimentar el empirismo y a partir de allí, se avanza continuamente.
- Usar historias y modelos para mostrar que construir → la parte de desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio ágil de “técnicos y no técnicos trabajando juntos”. Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo.

BRUF – Big Requirements Up Front

En una primera instancia de un proyecto de SW, desarrollar solo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.

Además, es prácticamente imposible saber al 100% todas las necesidades de los usuarios, porque ni ellos las saben. Por esto, y por lo cambiante de los requerimientos, la priorización es un factor fundamental en esa etapa, donde el equipo, en conjunto con el Product Owner, deberán ser capaces de decidir cuales son aquellos requerimientos indispensables para el negocio, de manera que, si se implementan, se puede entregar una nueva versión del producto que sea de valor para el cliente.

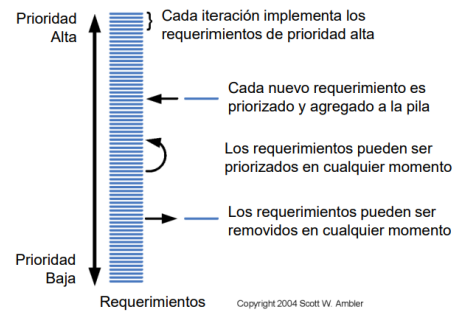


Product Backlog → lista priorizada de características y requerimientos de SW. Se almacenan priorizadas y estimadas las User Stories. Es el PO quien se encarga de definirla, ya que es él quien tiene en claro cuales son sus necesidades y requerimientos principales. El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.

En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado. Con esta situación se permite la compensación de no tener documentos de requerimientos formales escritos, detallando todos los puntos. Ágil ofrece una compensación al afirmar que la mejor comunicación es el cara a cara y un cliente participe en el equipo.

Gestión Ágil de Requerimientos de Software

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



Requerimientos Just in Time

Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. El Product Backlog nunca está al 100%

En el enfoque tradicional se aplica el “Up Front Specification”, mediante el cual se especifica todos los requerimientos antes de comenzar el desarrollo. Esto implica que un error en la captura de requerimientos tiene un alto impacto. En cambio, los requerimientos en ambientes ágiles al ser consensuado continuamente con el cliente, el impacto que se tiene es menor.

El producto no va a estar especificado al 100% desde el principio ni muchísimo menos, sino que se van a ir encontrando requerimientos y describiendo los requerimientos que hacen falta conforme haga falta. “ni antes, ni después”.

Tipos de requerimientos

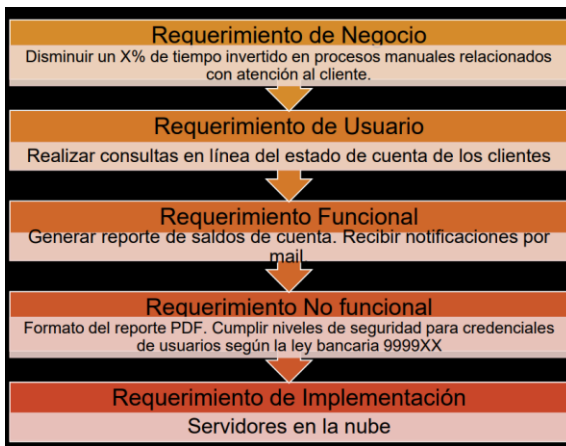
La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

A nivel de la gestión ágil, se trabaja con requerimientos de negocio y de usuario. Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.

Al usuario le interesa el requerimiento de negocio. El valor de negocio es el requerimiento de negocio, pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio. Las User Stories no sirven para especificar requerimientos de software, sirven para identificar requerimientos de usuario.

Debemos asumir que:

- Los cambios son la única constante.
- Stakeholders → no son todos los que están.
- Siempre se cumple que “El usuario dice lo que quiere cuando recibe lo que pidió”
- No hay técnicas ni herramientas que sirvan para todos los casos.
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar un resultado, una solución de “valor”.



La triple restricción – enfoque ágil

En el enfoque ágil, se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este. Entonces, el enfoque trata de dejar el tiempo y los recursos fijos, teniendo en cuenta “cuánto software” le puedo entregar en tanto tiempo y con tal equipo de desarrollo.

Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia los recursos y el tiempo.



Principios ágiles relacionados con requerimientos ágiles



User Stories

Técnica para trabajar requerimientos de usuario en ambientes ágiles. Contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.

Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Las US son escritas por el Product Owner.

Se consideran verticales dentro del producto, y multipropósito, ya que cumplen diferentes funciones en el desarrollo de un SW:

- Representan una necesidad de usuario, de forma no detallada.
- Conforman una descripción del producto.
- Son utilizadas como ítems de planificación.

- Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
- Junto con el código ejecutable, permiten documentar el producto.
- Tiene valor de negocio explícito.
- Reemplaza a la ERS según estándar de la IEEE.

Partes de la User Story

- 1) **Tarjeta** → descripción de la historia, usada para planificar y como recordatorio. Tiene una sintaxis particular:

<<Frase Verbal>>
Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

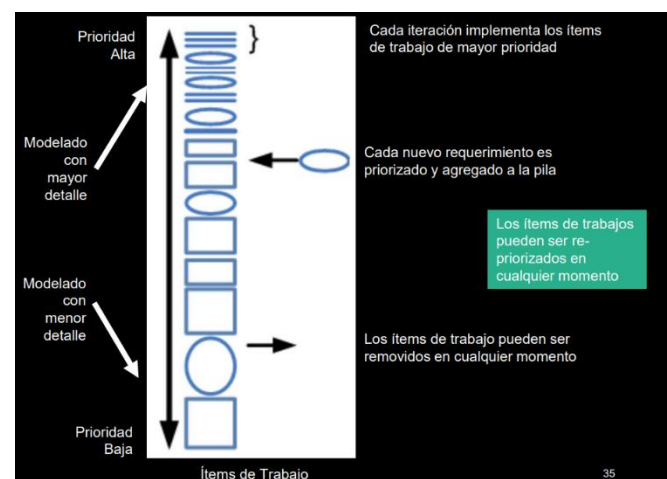
- **Nombre de rol** → representa a quien está haciendo la acción o quien recibe el valor de la actividad.
 - **Actividad** → representa la acción que realizara el sistema.
 - **Valor de negocio** → es la mas importante. Representa el por que es necesaria la actividad. Esto justifica la razón de que el PO escriba la US. Permite priorizar el desarrollo.
 - **Frase verbal** → es una forma corta de referenciar la US. Representa la funcionalidad expresada en la User.
 - **Criterios de aceptación** → son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los limites de la US para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad. Se deben redactar independientes a la implementación (alto nivel de abstracción).
 - **Pruebas de aceptación** → acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa. Las pruebas pueden agregarse o quitarse en cualquier momento.
- 2) **Conversación** → Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. En simples palabras, son discusiones acerca de la historia que sirven para desarrollar los detalles de la historia.
 - 3) **Confirmación** → Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conformar a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Product Backlog y las User Stories

Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

Porciones verticales

Las US son porciones verticales. Para agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de BD.



Modelado de roles

Se usan las tarjetas de rol de usuario. Aquí se anotan las características en general que va a tener el usuario. El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.

Técnicas adicionales:

- Personas → busca ser mas especifica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serian los objetos.
- Personajes extremos → que personas en términos extremos podrían llegar a utilizar el SW.
- Proxies (Usuarios Representantes) → es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio. No son ideales como usuarios verdaderos (EVITARLOS). Ej. Gerente de usuarios, gerente de desarrollo.

Criterios de aceptación → Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que este debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Lo fundamental es que esté escrito en términos objetivos y concretos.

- Definen límites para una US.
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos)
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad en una US.
- Sirve, en definitiva, para alinear al PO, testers y desarrolladores.

¿Cuáles son los criterios de aceptación buenos?

- Definen una intención, no una solución. Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- Son independientes de la implementación.
- Relativamente de alto nivel, no es necesario que se escriba cada detalle.

Detalles → son el resultado de las conversaciones con el PO y el equipo puede capturarlos en distintos lugares: documentación interna de los equipos; pruebas de aceptación automatizadas.

Pruebas de aceptación → Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación. El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen

Definition of Ready (DoR) – Definición de Listo → medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

INVEST Model → modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

- Independiente → la US es implementable en cualquier orden. No existe dependencia.
- Negociable → El cliente debe expresar lo que necesita, es decir que está centrada en el “qué” no el “cómo”. El cómo es decisión del equipo y debe ser negociable con el Product Owner. La US debe estar escrita en términos de qué necesita el usuario y no de cómo lo vamos a implementar.

- Valuable → Una US debe ser valorable para los clientes o usuarios (el para qué de la descripción).
- Estimable → la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo. Sin poder estimarla no es posible determinar si se puede finalizar en una iteración por lo tanto no puede ser construida (No puedo saber cuánto tardaría).
- Small (Pequeña) → la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración.
- Testable → la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada (se muestra que está hecha y demostrable).

Definition of Done (DoD) – Definición de Hecho → Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.

En la práctica consta de un Checklist de condiciones que deben cumplirse.

Algunos elementos de ese checklist son:

- Pruebas (unidad, integración, sistema) realizadas.
- Cero defectos conocidos y despliegue en producción.
- Código completo, refactorizado y con formato estándar.
- Código en repositorio e inspeccionado.

Puede ocurrir que una organización tenga un estándar de DoD, entonces los miembros del equipo deberán adaptarse a ese estándar. O bien, que la organización no posea un estándar, con lo cual el equipo deberá confeccionar y crear una Definición de Hecho adecuada para el producto.

Niveles de abstracción

- **User Story** → cumple con el criterio de listo (DoR) para ingresar a una iteración.
- **Épica** → es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User.

Una épica se suele identificar también cuando no cumple con el modelo de calidad de INVEST. La épica debe ser dividida en US más pequeñas.

Sirven para aquellos requerimientos que no son

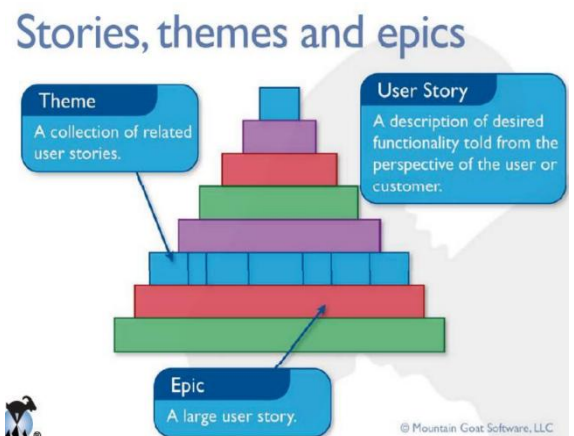
prioridades en las primeras iteraciones, y basta con tener sólo una aproximación general de las funcionalidades requeridas. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.

- **Temas** → son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Ambos (Épicas y Temas) permiten reducir los esfuerzos del equipo a la hora de estimar

Spikes → Son un tipo especial de US que se producen por la incertidumbre que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US.

Se clasifican en:



- **Técnicas** → utilizadas para investigar enfoques técnicos en el dominio de la solución. Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.
 - Evaluar performance potencial
 - Decisión hacer o comprar
 - Evaluar la implementación de cierta tecnología
- **Funcionales** → Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema. Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Algunas US requieren de ambos tipos de spikes.

Pueden utilizarse para:

- Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
- Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
- Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
- Frente a riesgos funcionales, donde no está claro como el sistema debe resolver la interacción con el usuario para alcanzar el beneficio esperado.

Lineamientos para Spikes

- Estimables, demostrables y aceptables
- La excepción, no la regla.
 - Toda historia tiene incertidumbre y riesgos.
 - El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
 - Los spikes deben dejarse como incógnitas mas criticas y grandes.
 - Utilizar spikes como ultima opción.
- Implementar la spike en una iteración separada de las historias resultantes.
 - Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.



Estimaciones de Software

En ambientes tradicionales

Estimación → predicción acerca del futuro realizada mediante alguna técnica. Por definición, no es exacta, porque se realiza con información incompleta y existe incertidumbre. Su objetivo no es “adivinar” el resultado final, sino brindar una base razonable para tomar decisiones y planificar.

No hay que confundir los siguientes conceptos:

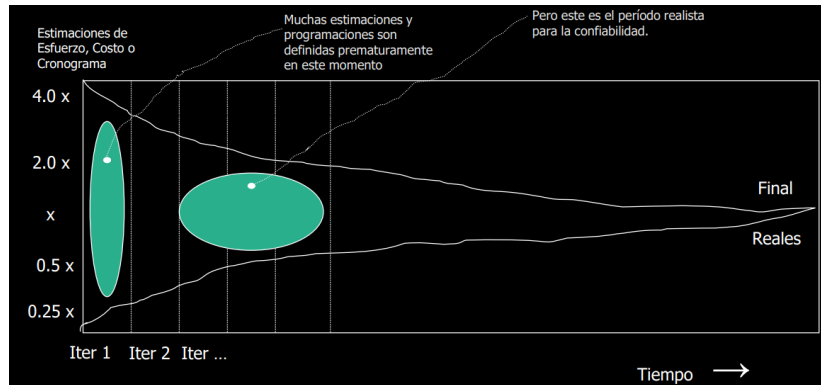
- **Estimación** → predicción de cuánto puede costar o demandar un trabajo según la información disponible.
- **Planificación** → utiliza las estimaciones como entrada para decidir cómo organizar el proyecto.
- **Objetivo** → resultado que el negocio desea alcanzar, por ejemplo, una fecha determinada.
- **Compromiso** → promesa acordada de entrega, que se asume considerando determinados riesgos.

Por lo tanto, estimar no es planificar y una estimación no es un compromiso. El plan puede ser distinto de la estimación, pero cuanto mayor sea esa diferencia, mayor será el riesgo asumido.

¿Para qué estimamos?

Al comienzo de un proyecto existe poca información sobre requisitos, tecnología, solución, riesgos, etc., por lo que las estimaciones presentan una gran incertidumbre. A medida que el proyecto avanza y se obtiene evidencia real, las estimaciones pueden refinarse y su rango disminuye.

Por eso, la estimación es un proceso iterativo y no estático: se realiza una estimación inicial y luego se revisa a medida que se aprende. Una estimación al final del proyecto no sirve de nada. Hay una estimación inicial y a partir de ahí se va refinando.



¿Qué podemos estimar?

En un enfoque tradicional, pueden estimarse medidas como:

- **Tiempo** → duración calendario.
- **Esfuerzo** → cantidad de trabajo requerida, por ejemplo, horas-persona.
- **Tamaño** → magnitud del producto o trabajo.
- **Recursos** → personas, herramientas u otros recursos necesarios.

Estas son medidas absolutas.

Errores de estimación

Las estimaciones pueden desviarse porque trabajamos con incertidumbre, pero algunos errores aumentan innecesariamente esa desviación. Por ejemplo: no utilizar técnicas, depender de una única técnica y no revisar las estimaciones a medida que cambia la información. Por eso conviene combinar técnicas y refinar las estimaciones durante el proyecto.

Son frecuentes dos problemas:

- Optimismo excesivo → subestimar sistemáticamente el trabajo.
- Actividades omitidas → estimar solamente la actividad principal y olvidar requerimientos, documentación, revisiones, testing, etc.

Técnicas de estimación

Existen distintos tipos de métodos:

- Basados en la experiencia
 - Datos históricos
 - Juicio experto
 - Puro
 - Delphi
 - Analogía
- Basados en recursos
- Basados en el mercado
- Basados en los componentes del producto o proceso de desarrollo
- Algorítmicos

Contar → técnica más elemental: identificar y contar elementos que puedan medirse objetivamente, utilizando luego esa cantidad como base para la estimación. Cuando algo puede contarse en lugar de estimarse subjetivamente, conviene hacerlo porque reduce incertidumbre.

Datos históricos → Consiste en utilizar información de proyectos o trabajos anteriores para estimar uno nuevo. El problema es que los datos históricos deben ser comparables y estar correctamente almacenados. Un proyecto anterior realizado con otro equipo, tecnología o contexto puede no ser una referencia adecuada.

Juicio experto → Una persona con experiencia analiza las especificaciones y genera una estimación utilizando su conocimiento previo. Es uno de los métodos más utilizados, pero presenta una debilidad: la calidad de la estimación depende mucho del experto. Si ese conocimiento está concentrado solamente en una persona, la organización depende excesivamente de ella. Tiene dos variantes:

- **Puro** → El experto analiza el trabajo y entrega directamente su estimación. Es sencillo, pero muy dependiente de su experiencia personal.
 - Optimista, habitual y pesimista → es una manera de estructurar el juicio experto, pensando tres posibles escenarios: optimista (O), habitual o más probable (H), y pesimista (P). Se usa la formula:

$$Estimación = \frac{O + 4H + P}{6}$$

El escenario habitual recibe mayor peso. Esto obliga al estimador a considerar explícitamente la incertidumbre en vez de pensar solamente en un único resultado.

- **Wideband Delphi** → Es una técnica de juicio experto grupal. En lugar de depender de una sola persona, varios expertos realizan estimaciones y las refinan mediante iteraciones hasta lograr una convergencia razonable. El proceso general es:
 - Se entregan las especificaciones al grupo.
 - Los expertos discuten el producto y aclaran dudas.
 - Cada uno realiza una estimación individual.
 - Un coordinador recopila las estimaciones.
 - Los participantes conocen las estimaciones de los demás de manera anónima.
 - Se discuten las diferencias.
 - Cada experto vuelve a estimar.
 - Se repite hasta lograr una convergencia razonable.

No es una votación democrática → No se elige simplemente el valor más votado. Las diferencias permiten descubrir supuestos, conocimientos o interpretaciones diferentes y, mediante la discusión, mejorar la estimación.

Analogía → Se estima un trabajo comparándolo con otro conocido. Es especialmente importante en estimaciones ágiles, porque los Story Points se asignan mediante comparaciones relativas.

Estimaciones en ambientes ágiles

No ser dogmático, ser pragmático → no existe una única técnica válida para todos los contextos. El equipo debe usar la práctica que mejor le sirva según el problema, el nivel de incertidumbre y la información disponible. Estimar no consiste en aplicar una receta rígida, sino en usar herramientas que ayuden a tomar mejores decisiones.

Matriz de desambiguación

- Objetivo → meta deseada por el negocio. Es algo que se quiere lograr, no necesariamente algo que técnicamente sea posible.
- Estimación → predicción analítica del esfuerzo, costo o duración probable según la información disponible.
- Compromiso → acuerdo de entrega que se asume considerando ciertos riesgos.

Lo importante es que la estimación no debe modificarse para que coincida con el objetivo del negocio. Si el negocio quiere una fecha determinada y la estimación indica otra, debe negociarse el compromiso: alcance, recursos, fecha o riesgo.

Subestimación → Ocurre cuando se fuerza una estimación artificialmente baja para hacerla coincidir con una fecha o expectativa del negocio. El ciclo es:

- 1) Se impone un cronograma irreal.
- 2) Para intentar cumplirlo, se recortan actividades tempranas, como análisis o diseño.
- 3) Se desarrollan soluciones rápidas y de menor calidad.
- 4) Aparecen mas defectos durante testing o producción.
- 5) Finalmente surgen reuniones de crisis, retrabajo, refactorización tardía y desgaste del equipo.

Subestimar para “ir más rápido” termina generando más costo, defectos y demora.

¿Cómo debería ser una buena estimación?

- Se expresa como rango, no como un único valor.
- Explicita un nivel de confianza o probabilidad.
- Se mantiene separada del objetivo de negocio → No se cambia una estimación solamente porque exista una fecha deseada
- Sirve para tomar decisiones.

Por eso, la utilidad de estimar no está solo en obtener un número, sino en hacer visible la situación del proyecto para poder decidir.

Tips respecto de las estimaciones

- Si una estimación se transforma directamente en un compromiso, puede resultar perjudicial.
- Muchas veces el mayor valor esta en el proceso de estimar, porque obliga al equipo a discutir el trabajo, descubrir incógnitas y hacer explícitos supuestos.
- Permite evaluar tempranamente si un trabajo o una fecha son factibles.
- Puede proteger al equipo frente a expectativas poco realistas, porque brinda evidencia para discutir fechas, alcance y riesgos.

Estimaciones ágiles

La principal diferencia es que, en ágil, normalmente no intentamos estimar directamente horas o días, sino el tamaño relativo del trabajo.

Las features o User Stories se comparan entre sí utilizando normalmente Story Points.

Estimación relativa → las personas suelen tener dificultades para estimar valores absolutos, pero somos mejores realizando comparaciones. La estimación relativa favorece la discusión grupal y permite utilizar el tiempo de estimación para comprender mejor las historias.

Tamaño → en ágil se utiliza como medida relativa. El tamaño es una medida de la cantidad de trabajo necesaria para producir una feature/story. Intenta representar cuan grande es una US en comparación con otras, teniendo en cuenta aspectos como:

- Cantidad de trabajo
- Complejidad
- Incertidumbre o duda/riesgo

Tamaño NO ES esfuerzo → el tamaño describe que tan grande resulta el trabajo en términos relativos. El esfuerzo representa lo que realmente cuesta ejecutarlo para determinadas personas. Por eso las estimaciones basadas directamente en tiempo pueden verse afectadas por habilidades, experiencia, conocimiento y familiaridad con el dominio.

Escalas de tamaño → se debe elegir una, y una vez elegida, esta no cambia. Es la unidad de medida. Pueden usarse por ejemplo, valores del 1 al 10, talles, potencias de 2, Fibonacci.

Story Point → Un Story Point (SP) es una unidad de medida relativa utilizada por un equipo para expresar el tamaño o “peso” de una User Story considerando su complejidad, trabajo requerido e incertidumbre. Los Story Points son además específicos del equipo. La complejidad de una feature/story tiende a incrementarse exponencialmente.

Convertir SP directamente a horas para comprometer fechas puede generar confusión entre tamaño y esfuerzo, un uso indebido de estimaciones, presión indebida sobre el equipo, y una pérdida del enfoque empírico.

Una de las escalas más utilizadas es una versión de la serie de **Fibonacci**: 1, 2, 3, 5, 8, 13, 21...

La separación entre los números aumenta a medida que crece el tamaño porque también aumenta la incertidumbre. Es mucho más difícil distinguir con precisión entre dos historias muy grandes que entre dos historias pequeñas.

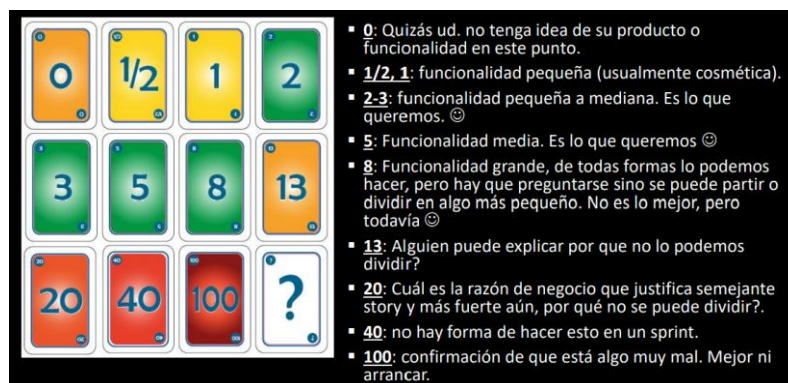
Velocidad → indica cuantos SP logra completar un equipo durante una iteración. Solo se cuentan historias completamente terminadas.

Su utilidad principal es empírica: después de varias iteraciones, permite conocer aproximadamente cuánto trabajo puede completar realmente ese equipo y utilizar esa información para proyectar.

Corrige los errores de estimación, porque aunque la escala inicial no sea perfecta, al relacionarla con la capacidad real observada del equipo podemos hacer previsiones progresivamente mejores.

Poker Estimation → técnica colaborativa para estimar US. Está basada en Wideband Delphi y combina juicio experto, analogía y desagregación.

Quienes estiman son fundamentalmente las personas que realizarán el trabajo, porque son quienes poseen el conocimiento necesario para evaluar su complejidad.



Historia canónica → se selecciona una User Story de referencia o canónica, cuyo tamaño sea bien comprendido por el equipo. Por ejemplo, se le puede asignar 1 SP. Las demás se estiman comparándolas con ella. Esta historia funciona como el patrón de medida del equipo.

Funcionamiento del Poker Estimation

- 1) El PO presenta la historia.
- 2) El equipo discute y aclara las dudas necesarias.
- 3) Cada estimador elige en privado una carta.
- 4) Todos muestran las cartas simultáneamente.
- 5) Si coinciden, se adopta ese valor.
- 6) Si existen diferencias, principalmente quienes eligieron los valores mas alto y mas bajo explican sus razones.
- 7) Se vuelve a estimar.
- 8) Se repite hasta alcanzar una convergencia razonable.

Mostrar simultáneamente las cartas evita que quien habla primero ancle las estimaciones de los demás. Lo más importante tampoco es llegar rápidamente al número. La discusión es una de las principales ventajas de la técnica, porque una diferencia grande suele revelar que alguna persona observó una complejidad, requisito, riesgo o supuesto que las demás no habían considerado.

¿Cuándo reestimar? → no se debería reestimar una tarea solo porque termino llevando mas tiempo del esperado. Si el tamaño relativo de la historia era correcto pero el equipo fue más lento, tuvo interrupciones o encontró circunstancias operativas, ese efecto debe reflejarse en la velocidad. Se reestima cuando descubrimos que nuestra comprensión original del tamaño relativo era incorrecta; por ejemplo, aparece una complejidad o tecnología desconocida que cambia significativamente la naturaleza de la historia.



Negocia el compromiso, nunca la estimación → una estimación es un cálculo basado en datos; negociarla es tan inútil como negociar la temperatura del sol. Lo que si se negocia es el compromiso: ajustar el alcance, mover la fecha o aceptar más riesgo de negocio.

Relación final: tamaño, velocidad y duración

En ágil estimamos tamaño y obtenemos duración a partir de la velocidad empírica del equipo. Si un backlog tiene 100 SP y el equipo demuestra una velocidad aproximada de 20 SP por sprint: $100 / 20 = 5$ iteraciones. Así pasamos de una estimación relativa de tamaño a una posible duración utilizando datos reales del equipo. Inicialmente estimamos, pero después inspeccionamos lo que realmente ocurre, adaptamos las previsiones y volvemos a iterar.



Eligiendo la Unidad: Puntos de Historia vs. Días Ideales

Puntos de Historia (Story Points)	Días Ideales (Ideal Days)
Naturaleza	
Medida pura de tamaño y complejidad.	Tiempo ideal de trabajo sin interrupciones.
Efecto del Tiempo (Decay)	
No caducan. Un 'Labrador' siempre será un 'Labrador' aunque el equipo mejore.	Se devalúan a medida que el equipo se vuelve más rápido o experto.
Impacto en el Equipo	
Fomenta el consenso cross-functional ('somos un solo equipo').	Fomenta el sesgo individual ('mis días no sons días').

Recomendación: Usar Puntos de Historia para separar completamente el esfuerzo de la duración.

Story Points vs. Días Ideales

Ideales: los SP expresan tamaño relativo y no tiempo, mientras que los días ideales expresan cuánto demoraría el trabajo sin interrupciones. Se prefieren los SP porque permiten separar el tamaño de la duración y son menos dependientes de la velocidad particular de una persona.

Scrum: el Framework

Es un marco de trabajo liviano que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas para problemas complejos.

Framework ≠ Metodología → una metodología prescribe con mayor detalle que pasos seguir y como realizarlos. Un framework, por otro lado, establece una estructura mínima de roles/responsabilidades, eventos y artefactos, pero deja libertad para decidir que técnicas concretas usar dentro de ese marco.

Scrum es un framework → establece una estructura mínima, dentro de la cual el equipo puede elegir las prácticas y técnicas que considere necesarias.

Fundamento de Scrum → se apoya en dos ideas: empirismo y pensamiento Lean (eliminar desperdicio). Además, utiliza un desarrollo iterativo e incremental, para aprender y controlar el riesgo.

Principios de Scrum

- **Control empírico** → se toman decisiones según evidencia y experiencia, no suponiendo que todo puede predecirse.
- **Autogestión** → el equipo decide internamente como organizar y realizar su trabajo.
- **Colaboración** → el conocimiento y decisiones surgen de la interacción entre miembros e interesados.
- **Priorización por valor** → se trabaja primero sobre aquello que genera mayor valor. Se aplica principalmente en el Product Backlog.
- **Time-boxing** → los eventos tienen duraciones limitadas para generar ritmo, foco y oportunidades frecuentes de feedback.
- **Desarrollo iterativo** → el producto evoluciona mediante ciclos sucesivos de construcción, inspección y adaptación.

La lente OODA → OODA describe un ciclo para tomar decisiones en contextos complejos: Observe → Orient → Decide → Act.

- **Observar** → reunir evidencia de lo que realmente está ocurriendo.
- **Orientar** → interpretar esa realidad y darle contexto.
- **Decidir** → elegir que hipótesis o experimento intentar.
- **Actuar** → ejecutar la decisión y volver a observar resultados.

No reemplaza los pilares del empirismo. Más bien muestra cómo se materializa el aprendizaje y la decisión dentro del ciclo empírico. Se pueden relacionar:

Observar/Orientar → Transparencia + Inspección

Decidir/Actuar → Adaptación

Valores de Scrum → se vinculan con OODA (este último sirve para ordenar los valores de Scrum).

- **Apertura + Respeto** -> Observar → necesitamos evidencia honesta y escuchar otras perspectivas.
- **Coraje** -> Orientar → aceptar la realidad y cuestionar supuestos, aunque sea incomodo.
- **Foco** -> Decidir → elegir que es prioritario y no dispersarse.
- **Compromiso** -> Actuar → ejecutar lo acordado y avanzar hacia el objetivo (Sprint Goal).

Sin Coraje para decir "no" y Respeto para criticar ideas (no personas), el empirismo falla.

Scrum Team como "célula profesional" → se usa la analogía de una célula para mostrar tres características centrales del equipo:

- **Autogestión** → el equipo decide quién hace qué, cuándo y cómo, sin microgestión externa.
- **Profesionalismo** → el equipo asume responsabilidad sobre la calidad y el ciclo de vida del producto. Excelencia técnica y ética.
- **Diversidad cognitiva** → se combinan distintas habilidades, experiencias y puntos de vista para resolver problemas complejos.

El Scrum Team es la unidad fundamental de Scrum. Es un equipo pequeño (se recomienda de 10 personas o menos, para mejor comunicación), multifuncional y autogestionado, con todas las habilidades necesarias para generar valor. No posee subequipos ni jerarquías internas y comparte la responsabilidad por el resultado.

Ecosistema extendido: Stakeholders y Supporters

Stakeholders → personas o entidades interesadas o afectadas por el producto: clientes, usuarios, patrocinadores, etc. Aportan necesidades, requisitos, feedback y validación de valor. Entidades no humanas, como la IA, también se encuentran aquí.

Supporters → concepto nuevo de la expansión 2026. Stakeholders con capacidad de ayudar al Scrum Team a funcionar dentro de la organización: líderes o gerentes que eliminan barreras organizacionales, proveen recursos y generan un entorno favorable para la autogestión. Ayudan a “insertar” al Scrum Team dentro de una organización y actúan como intermediarios frente a impedimentos organizacionales.

Roles en Scrum → hay 3 responsabilidades dentro del Scrum Team:

Product Owner → responsable de maximizar el valor del producto a largo plazo y de gestionar eficazmente el Product Backlog y la narrativa del producto. Entre sus responsabilidades están:

- definir y comunicar el Product Goal
- asegurar que los elementos del Product Backlog sean claros
- ordenarlos según valor
- mantener el backlog visible y comprensible.

Se ocupa del “Qué” y el “Por qué” del producto. Decide que problema conviene atacar y por que aporta valor, pero no le indica a los developers como implementarlo.

Developers / Product Developers → son quienes crean un Increment utilizable y de calidad en cada Sprint.

Incluye humanos y agentes automatizado. Responsables de:

- crear el Sprint Backlog;
- determinar cómo realizar el trabajo;
- cumplir la Definition of Done, garantizando la calidad;
- adaptar diariamente su plan para alcanzar el Sprint Goal;
- responsabilizarse mutuamente como profesionales.

Son el “Cómo” + Calidad.

Scrum Master → no es un jefe que distribuye tareas ni controla individualmente al equipo. Su responsabilidad principal es mejorar la efectividad del Scrum Team y ayudar a que Scrum sea comprendido y aplicado correctamente. Actúa como facilitador, líder servicial, coach, agente de cambio y facilitador de la eliminación de impedimentos. También trabaja hacia afuera del equipo, ayudando a la organización a adoptar Scrum y eliminando barreras entre el equipo y los stakeholders. Es = Efectividad.

Stakeholders y Supporters están dentro del ecosistema extendido, pero no forman parte del Scrum Team.

IA en Scrum 2026

Como ya mencioné, un Developer puede ser humano o automatizado, incorporando IA como una especie de “compañero de equipo”, capaz de colaborar en actividades como:

- automatización;
- generación;
- análisis;
- detección de patrones;
- apoyo al desarrollo.

Sin embargo, aparece una condición fundamental:

Human in the Loop → La IA aumenta la capacidad, pero la responsabilidad (accountability) permanece 100% en los humanos. Esto implica:

- supervisar los resultados;

- validar decisiones;
- utilizar IA éticamente;
- mantener transparencia;
- reducir sesgos;
- no delegar completamente el juicio profesional.

Liderazgo y cambio → En contextos complejos, liderar no significa imponer un plan detallado e inmutable, sino crear las condiciones para que el equipo pueda observar, aprender y adaptarse.

Liderazgo para la resiliencia → El liderazgo es una conducta, no solamente un cargo. El líder ayuda a:

- crear claridad;
- mantener un entorno seguro;
- favorecer decisiones basadas en evidencia;
- permitir que el equipo se adapte.

El cambio es emergente → En un problema complejo no puede conocerse desde el principio todo el camino hasta la solución. Por eso se mantiene una dirección, pero no necesariamente un camino fijo. Los líderes actúan como catalizadores del cambio, no como personas que predicen exactamente todo lo que ocurrirá.

Genchi Genbutsu → principio proveniente de Lean que propone ir al lugar donde ocurre el trabajo, observar directamente y tomar decisiones basadas en hechos reales en lugar de únicamente en reportes o suposiciones.

Sprint → es el contenedor de todos los demás eventos Scrum y constituye el ciclo básico de trabajo. Tiene una duración fija de un mes o menos, y un nuevo Sprint comienza inmediatamente después del anterior. Es el corazón del ritmo y la cadencia.

La cadencia es importante, porque la duración estable genera:

- previsibilidad;
- ritmo de trabajo;
- oportunidades regulares de inspección;
- ciclos frecuentes de aprendizaje.

Cuanto más corto el Sprint, menor es el costo potencial del error. El sprint entonces funcionaría como un contenedor para poder probar y fallar (safe to fail). Nos da estabilidad dentro del caos.

La cadencia es fija → queremos que el cliente tenga predecibilidad.

Durante el Sprint:

- No se realizan cambios que pongan en peligro el Sprint Goal. Es inamovible.
- El alcance puede aclararse y renegociarse entre Developers y Product Owner, en caso de aparecer nueva información.

No hay “Sprints de Hardening” → No existe la idea de hacer primero el desarrollo “rápido” y dejar un Sprint de hardening para solucionar calidad después. Cada Sprint debe generar una salida de calidad. La calidad nunca debe disminuir para cumplir el Sprint.

Eventos dentro el Sprint → cada uno representa una oportunidad de inspección y adaptación.

Sprint Planning → evento que inicia el Sprint. Responde a:

- **¿Por qué?** → Se define el Sprint Goal, es decir, por qué ese Sprint aporta valor.
- **¿Qué?** → Los Developers seleccionan junto con el PO qué elementos del Product Backlog pueden realizarse.
- **¿Cómo?** → Los Developers planifican cómo transformar esos elementos en un Increment que cumpla la Definition of Done.

El resultado de este Sprint Planning es el **Sprint Backlog** (Sprint Goal + PBI seleccionados + plan de trabajo).

El equipo debe planificar holgura para manejar la incertidumbre.

Daily Scrum → reunión de 15 minutos de los Developers para los Developers. Su propósito es: inspeccionar el progreso hacia el Sprint Goal y adaptar el Sprint Backlog para el siguiente día de trabajo. Puede utilizar las clásicas preguntas “qué hice, qué haré y qué impedimentos tengo”, pero NO ES un reporte de estado para el Scrum Master.

Sprint Review (Validación de valor) → inspecciona el resultado del Sprint junto con stakeholders para determinar qué adaptar en el producto. Es una sesión de trabajo colaborativa, donde se analiza:

- qué se logró;
- qué cambió en el entorno;
- qué feedback existe;
- qué conviene hacer después.

El Product Backlog puede modificarse por el aprendizaje. Validación contra la definición de Outcome Done.

Sprint Retrospective → no analiza principalmente el producto, sino cómo trabajó el equipo. El objetivo es aumentar la calidad y efectividad del Scrum Team.

Timeboxes → todos estos eventos tienen duración máxima.



Se limita el tiempo para generar foco, cadencia y previsibilidad. Si no entra todo el alcance, no se alarga arbitrariamente el Sprint: se protege el tiempo y se adapta el alcance.

Cambio fundamental: de salida a impacto

- **Increment / Output** → lo que construimos. Representa una salida técnicamente terminada: un increment utilizable que cumple estándares de calidad. ¿Lo construimos correctamente?
- **Product / Outcome** → el resultado o impacto que conseguimos con eso. El efecto observable que la salida produce en los usuarios o el negocio. ¿Construimos lo correcto? ¿Generamos valor?

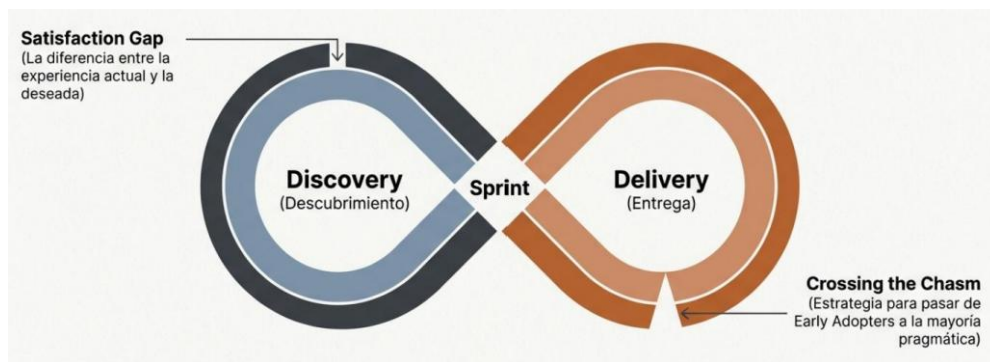
El valor sigue siendo una suposición hasta que se valida con evidencia. Es decir, terminar una funcionalidad no demuestra por sí solo que era valiosa.

Product Thinking

Project Thinking vs. Product Thinking → Un proyecto es temporal: comienza, busca alcanzar ciertos objetivos y termina. En cambio, un producto es un activo vivo que continúa evolucionando mientras siga aportando valor. Por eso, Scrum 2026 propone pensar más allá de “terminar el proyecto”, y mirar la evolución completa del producto:

- **Product Vision** → representa el norte general o estado deseado del producto.
- **Product Goal** → convierte esa visión amplia en un objetivo estratégico concreto que orienta el trabajo.
- **Product Backlog** → contiene el trabajo emergente necesario para avanzar hacia el objetivo y se ordena según valor.

Scrum gestiona el ciclo de vida completo, desde la tuna hasta la tumba.



Son un ciclo continuo, no fases separadas. El Product Discovery ocurre dentro del sprint. No hacemos primero meses de análisis y después meses de construcción. Mientras entregamos, seguimos aprendiendo sobre qué deberíamos construir.

No trata solamente del **Delivery** → construir y entregar funcionalidades.

También existe **Discovery**, que consiste en descubrir:

- qué problema conviene resolver;
- qué necesitan realmente los usuarios;
- qué hipótesis vale la pena probar;
- qué solución puede generar valor.

Artefactos y Compromisos

Artefacto	Compromiso
Product Backlog Lista emergente y ordenada de lo que se necesita.	Product Goal El objetivo estratégico a mediano plazo (El 'Por qué' estratégico).
Sprint Backlog El plan de los Developers para el Sprint.	Sprint Goal El objetivo único y coherente del Sprint (El 'Por qué' táctico).
Increment El ladrillo concreto de valor entregado.	Definition of Done El estándar de calidad requerido para liberar.

El compromiso da dirección al artefacto.

Doble definición de Done: Output vs. Outcome

Definition of Output Done → se enfoca en la calidad técnica. ¿Construimos el producto correctamente?

Puede requerir código limpio y probado, integración, documentación y cumplimiento de estándares.

Debe alcanzarse dentro del Sprint para que el trabajo forme parte del Increment.

Definition of Outcome Done → se enfoca en el valor de negocio. ¿Construimos el producto correcto?

Se evalúa mediante cambios observables en el comportamiento del usuario, impacto de negocio, ROI o validación de hipótesis.

Muchas veces solo puede medirse después del lanzamiento, porque hace falta observar el uso real.

Output es el requisito previo; Outcome es el éxito real.

Refinamiento del Product Backlog → el PB al principio puede tener ideas difusas, hipótesis e incertidumbre.

Mediante refinamiento continuo, esas ideas se investigan, dividen, aclaran y dimensionan hasta conseguir suficiente claridad para un Sprint.



El refinamiento cumple 3 funciones:

- Reducir incertidumbre → entender progresivamente los PBI.
- Aceptar incertidumbre → investigar y validar en lugar de suponer que todo lo conocido es correcto.
- Priorizar por valor → el PO ordena pensando principalmente en valor y dirección a largo plazo, no solamente en urgencia.

Triple filtro de Scrum: Ready, Output y Outcome

Ready - ¿Hay claridad suficiente para empezar? →

busca que un PB Item tenga suficiente claridad y preparación como para poder incorporarse a un Sprint. No es un compromiso formal, sino una cualidad emergente del refinamiento. No es tener todo resuelto, sino haber reducido suficientemente la incertidumbre.

Output Done - ¿Lo construimos correctamente? → se concentra en la calidad técnica del Incremento. Busca garantizar que lo desarrollado cumpla los estándares necesarios independientemente de la presión de tiempo.

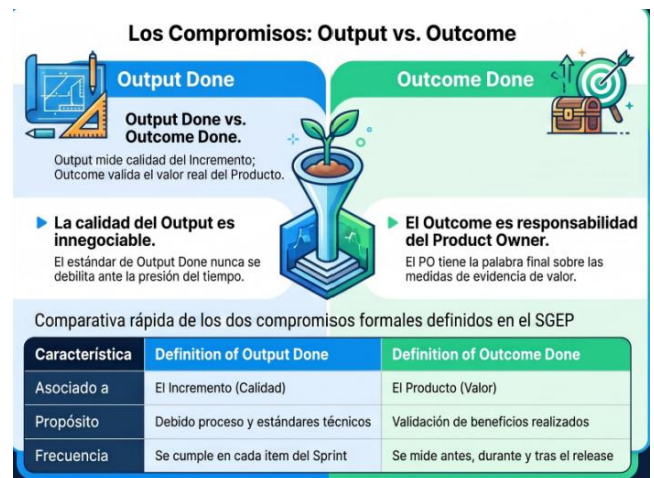
Outcome Done - ¿Lo construido produjo valor? → Se concentra en el resultado obtenido con el producto, no simplemente en que la funcionalidad haya sido terminada.

El Ready se conecta con el Que y el Por que → un PBI suficientemente preparado no debería indicar solamente qué hay que construir, sino también permitir comprender qué valor o resultado se espera conseguir.

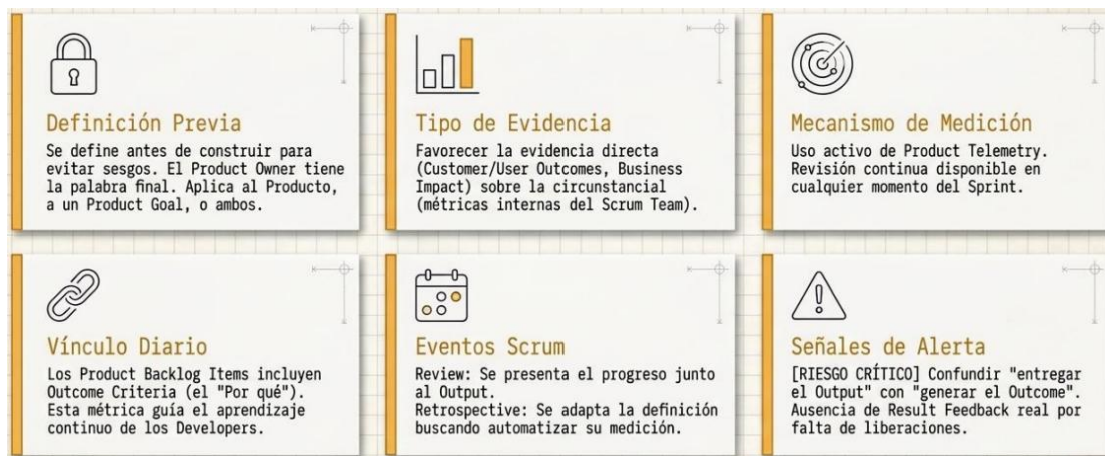
Definition of Ready → completo lo ya descrito con INVEST.

- **Claridad** → el PBI no debe contener ambigüedades importantes y debe conocerse qué se quiere conseguir y por qué.
- **Tamaño y factibilidad** → tiene que ser abordable por los Developers y deben conocerse o mitigarse sus principales dependencias.
- **Investigación previa** → si existe demasiada incertidumbre, puede ser necesario investigar, experimentar o hacer pruebas antes de planificar.
- **Colaboración activa** → PO, Developers y stakeholders colaboran para comprender el problema; el Product Owner no impone el “cómo”.
- **Coherencia con el rumbo** → el PBI debe estar relacionado con el Product Goal y poder contribuir a un Sprint Goal coherente.
- **Verificación final** → debe existir confianza suficiente para comenzar. Busca claridad suficiente, no absoluta.

Definition of Output Done



Definition of Outcome Done



Matriz de Calibración Estratégica

	Definition of Ready	Definition of Output Done	Definition of Outcome Done
Estatus en el SGEF	Cualidad emergente (Refinement).	Compromiso formal.	Compromiso formal.
Artefacto Asociado	Product Backlog Item (PBI).	Increment.	Product.
Responsabilidad y Decisión	Scrum Team colaborativo (Devs estiman).	Developers (Accountability colectiva).	Product Owner (Final say).
Métrica Típica	Días / Capacidad de comprensión.	Tests, Seguridad, Estándares técnicos.	Telemetría, Comportamiento de usuario, ROI.

Capacidad del equipo en un Sprint

Representa cuanto trabajo puede asumir razonablemente el equipo durante un Sprint. No equivale a sumar todas las horas disponibles de todas las personas, porque parte de esa capacidad se consume en otras actividades necesarias. La capacidad se usa en el Sprint Planning para decidir cuánto trabajo puede incorporar el equipo al Sprint Backlog, y puede expresarse en horas ideales o, en equipos más maduros, relacionarse con Story Points.

Buffer del sprint → margen de capacidad que no se compromete de antemano, para absorber incertidumbre e imprevistos.

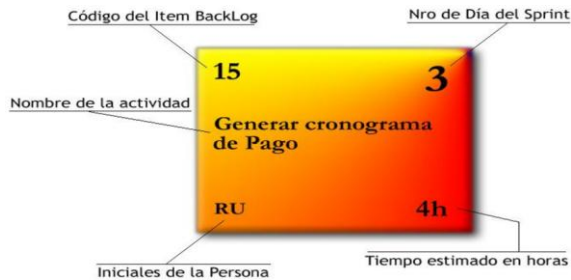


Ambiente de trabajo

- Abierto
 - Con comunicación visible y frecuente
 - El silencio absoluto es un mal signo → Scrum necesita interacción, colaboración y comunicación.
- Uso de pizarrones
- El equipo define sus horarios

Herramientas de Scrum

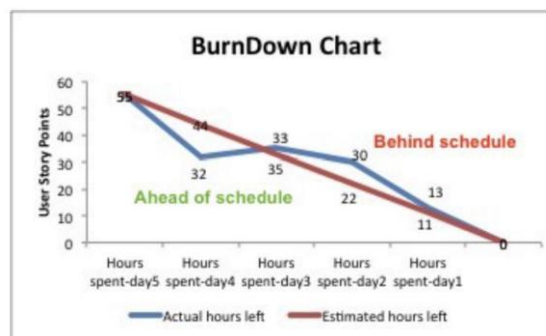
- **Tarjeta de tarea** → representa una tarea concreta derivada del trabajo del Sprint. Sirve para que el trabajo sea visible y fácil de seguir durante el Sprint.



- **Taskboard** → tablero visual donde se representa el estado del trabajo durante el Sprint.

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... DC 8
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4			Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 8
	Code the... 4	Code the... 6			Test the... SC 6

- Granularidad de las tareas → el trabajo relativo a una US debe dividirse en tareas mas pequeñas para que el avance sea más visible y controlable.
- **Sprint Burndown Chart** → muestra cuánto trabajo queda por realizar durante el Sprint. La curva ideal desciende progresivamente hasta llegar a cero al finalizar el Sprint. No mide valor, sino progreso del trabajo.



- **Gráficos de seguimiento del producto** → permiten observar progreso más allá de un único Sprint. Ejemplos: Release Burn Chart, Product Burndown, Burnup

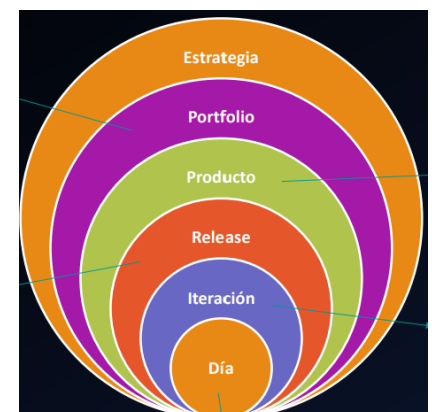
Múltiples niveles de planificación → la planificación no ocurre una única vez, sino en varios niveles.

Cuanto mas lejano es el horizonte:

- Hay menos detalle
- Mas incertidumbre
- La planificación es más estratégica

Cuanto más cercano:

- Aumenta el detalle
- Se usan datos más recientes
- Las decisiones son más operativas



¿Qué y cuando estimar? → tres niveles:

- **Portfolio Backlog** → se comparan productos o grandes iniciativas con estimaciones muy generales, por ejemplo tamaños relativos tipo S, M, L, XL.

- **Product Backlog** → las features/PBI se estiman con una granularidad intermedia, normalmente mediante Story Points.
- **Sprint Backlog** → las tareas concretas pueden estimarse con mucho más detalle, por ejemplo en horas.

No tiene sentido exigir precisión cuando todavía existe mucha incertidumbre.

Planificación de Portfolio → planificación estratégica que selecciona y prioriza productos o grandes iniciativas en las que la organización invertirá. Las propuestas pasan por criterios estratégicos y económicos para decidir si continúan (Go) o no (No-Go). Horizonte aproximado de un año o mas, involucrando principalmente a stakeholders y Product Owners, y generando como resultado un Portfolio Backlog.

Planificación de Producto → define la visión y evolución esperada del producto a mediano/largo plazo. Traduce una idea aprobada en una dirección de producto, un Product Backlog y una estrategia de releases.

Participantes

- stakeholders internos;
- Scrum Team;
- otras personas relevantes.

Entradas

- una idea inicial;
- una idea ya probada;
- horizonte de planificación;
- fechas esperadas;
- presupuesto/recursos;
- nivel de confianza disponible.

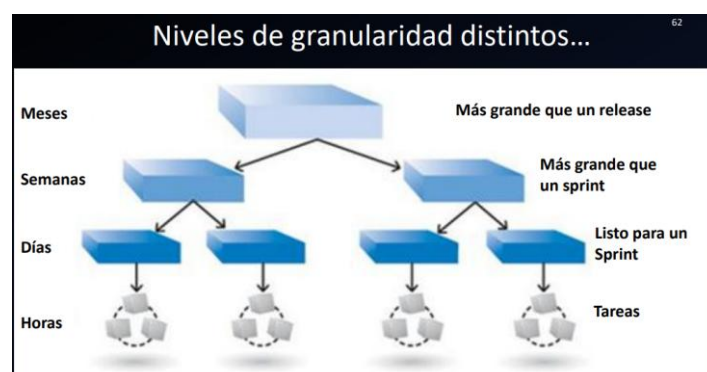
Actividades principales

- creación de la **Product Vision**;
- creación inicial del **Product Backlog**;
- elaboración del **Product Roadmap**;
- revisión e iteración de esos elementos.

Salidas

- Product Vision;
- Product Backlog inicial;
- Product Roadmap;
- otros artefactos necesarios.

Planificación de Release → determina qué funcionalidades se espera incluir en una entrega y aproximadamente cuántos Sprints serán necesarios. Combina prioridad del Product Backlog, tamaño estimado y velocidad del equipo, y se actualiza continuamente según la evidencia real. El resultado es un Release Plan. La velocidad permite transformar una estimación de tamaño del Product Backlog en una previsión de cuántos Sprints serán necesarios para una Release.



Release vs Sprint → Una Release puede contener varios Sprints. Cada Sprint genera un Increment utilizable, mientras que una Release representa la decisión de poner una determinada versión del producto a disposición de usuarios o clientes.

La frecuencia de liberación no tiene por qué coincidir con la cadencia de los Sprints.

Métricas ágiles → buscan aportar evidencia útil para inspección y adaptación. No se trata de medir todo lo posible, sino únicamente aquello que ayude a comprender el producto, el equipo o el progreso y permita tomar mejores decisiones.

- La medición es una salida, no una actividad → los datos deberían surgir naturalmente del trabajo realizado, evitando montar procesos burocráticos únicamente para producir indicadores.
- Medir lo necesario y nada mas

Principios ágiles que guían las métricas

- Satisfacer al cliente mediante entregas tempranas y continuas de software valioso.
- El software funcionando es la principal medida de progreso.

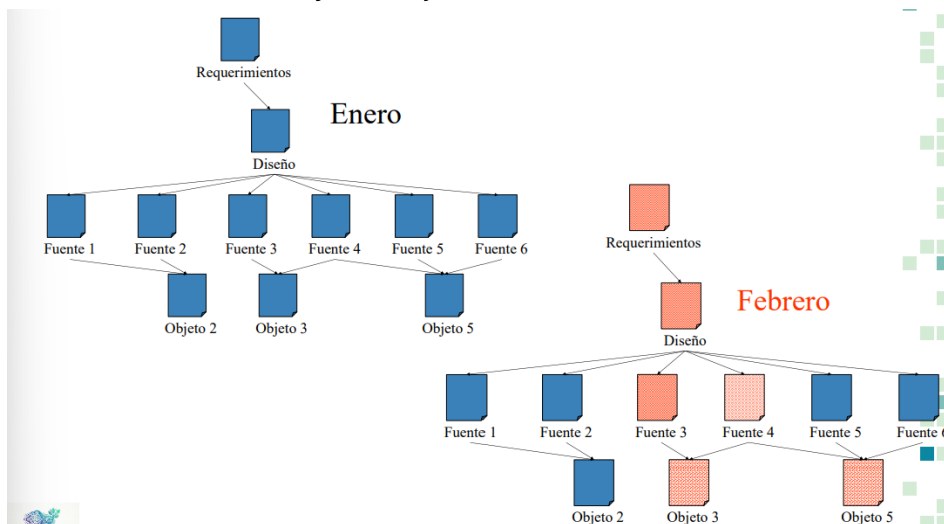
Esto orienta la elección de métricas hacia el producto y el valor, más que hacia indicadores burocráticos del proceso.

Ejemplos:

- Running Tested Features (RTF) → cantidad de funcionalidades completamente implementadas, probadas y funcionando. Permite medir avance real del producto, pero no considera el tamaño o complejidad de cada funcionalidad.
- Capacidad
- Velocidad

Software Configuration Management (SCM)

El software **evoluciona**, y está sujeto a **cambios**.



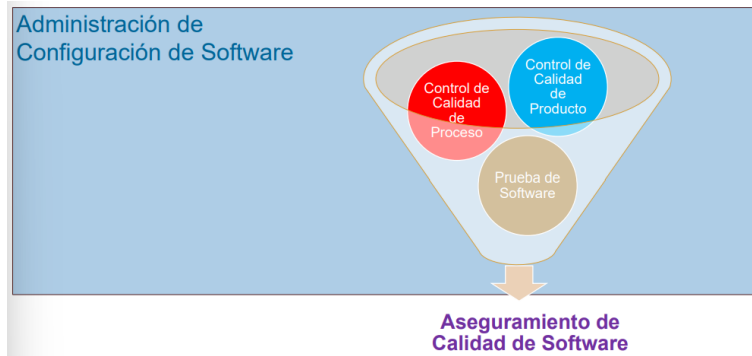
Los **cambios** en el SW tienen su origen en:

- Cambios del negocio y nuevos requerimientos
- Soporte de cambios de productos asociados
- Reorganización de las prioridades de la empresa por crecimiento
- Cambios en el presupuesto
- Defectos encontrados a corregir
- Oportunidades de mejora

Para manejar estos cambios, es que surge la **Gestión de Configuración de Software (SCM)**.

SCM como disciplina de soporte → es una actividad “paraguas”, transversal a todo el proyecto, relevante para el producto a lo largo de su ciclo de vida.

Disciplinas de soporte del SW



SCM – Definición → disciplina que aplica dirección y monitoreo administrativo y técnico a:

- identificar y documentar las características funcionales y técnicas de los ítems de configuración
- controlar los cambios de estas características
- registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos.

Propósito → establecer y mantener la integridad de los productos de SW a lo largo de su vida. Involucra para la configuración:

- Identificarla en un momento dado
- Controlar sistemáticamente sus cambios
- Mantener su integridad y origen.

Integridad del producto

- Satisface las necesidades del usuario
- Puede ser fácil y completamente rastreado durante su ciclo de vida
- Satisface criterios de performance
- Cumple con sus expectativas de costo

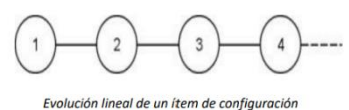
Problemas en el manejo de componentes

- Pérdida de un componente
- Pérdida de cambios (el componente que tengo no es el último)
- Sincronía fuente – objeto – ejecutable
- Regresión de fallas
- Doble mantenimiento
- Superposición de cambios
- Cambios no validados

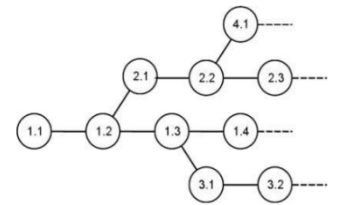
Conceptos

Ítem de Configuración de Software (SCI) → todos y cada uno de los artefactos que forman parte del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos saber su estado y evolución. Ejemplos: plan de SCM, visión, riesgos, requerimientos, arquitectura de SW, código fuente, casos de prueba, etc.

Versión → forma particular de un artefacto en un instante o contexto dado. El control de versiones refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado. La evolución puede representarse gráficamente en forma de grafo.



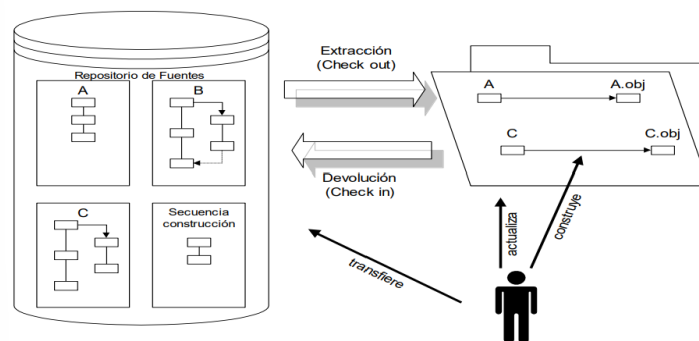
Variante → versión de un IC (o de la configuración) que evoluciona por separado. Representan configuraciones alternativas. Un producto de SW puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale.



Configuración del Software → un conjunto de ítems de configuración con su correspondiente versión en un momento determinado.

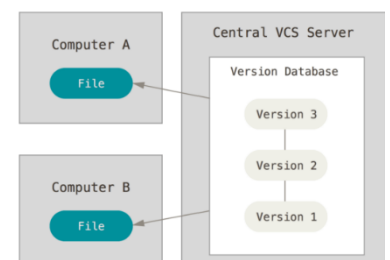
Repositorio → repositorio de información conteniendo los ítems de configuración (ICs). Mantiene la historia de cada IC con sus atributos y relaciones. Usado para hacer evaluaciones de impacto de los cambios propuestos. Pueden ser una o varias bases de datos.

Funcionamiento del repositorio



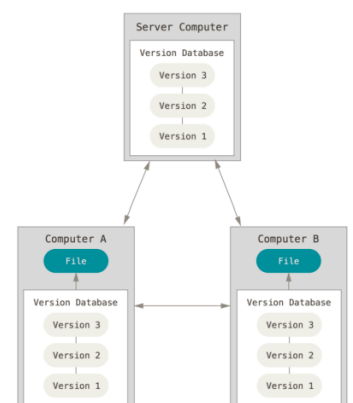
Repositorios Centralizados

- Un servidor contiene todos los archivos con sus versiones.
- Los administradores tienen mayor control sobre el repositorio.
- Falla el servidor y “estamos al horno”.



Repositorios Descentralizados

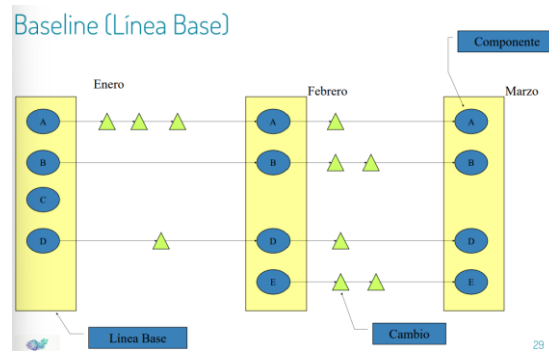
- Cada cliente tiene una copia exactamente igual del repositorio completo.
- Si un servidor falla solo es cuestión de “copiar y pegar”.
- Posibilita otros workflows no disponibles en el modelo centralizado.



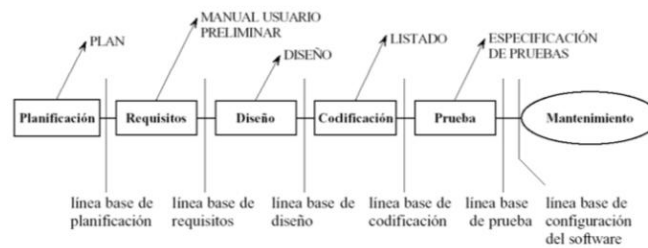
Líneas Base → configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo.

Sirve como base para desarrollos posteriores y puede cambiarse solo a través de un procedimiento formal de control de cambios. Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto. No debe confundirse con una versión del producto.

Identificación de la Línea Base → se usan etiquetas para marcar las baseline. No confundir con la versión del producto.



Representación de Líneas Base → pueden ser: de especificación (Requerimientos, Diseño) (no tienen código, quedan en la documentación); u operacionales: de productos que han pasado por un control de calidad definido previamente (contienen código).



Ramas

Creación de ramas

- Existe una rama principal (trunk, master)
- Sirven para bifurcar el desarrollo
- Pueden tener razones de creación con semántica
- Permiten la experimentación
- Pueden ser descartadas o integradas

Integración de ramas

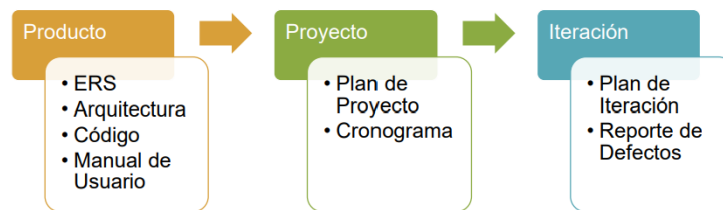
- La operación se llama merge
- Lleva los cambios a la rama principal
- Pueden surgir conflictos (resolvemos con diff)
- Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas

Actividades fundamentales de la Gestión de Configuración de Software

Identificación de ítems de configuración → permite distinguir los elementos que vamos a gestionar y establecer como se organizan y localizan.

- Identificación univoca de cada ítem de configuración → cada ítem debe tener una identificación que permita distinguirlo de otro, sin ambigüedades. También se debe poder reconocer su versión.
- Convenciones y reglas de nombrado → son criterios comunes para asignar nombres o códigos a los ítems y sus versiones. Evitan que cada integrante los nombre de una manera diferente y facilitan su reconocimiento.
- Definición de la Estructura del Repositorio → establecer cómo se organizarán los ítems dentro del repositorio, según las necesidades del proyecto.
- Ubicación dentro de la estructura del repositorio → determinar dónde se guarda cada ítem concreto dentro de la organización definida.

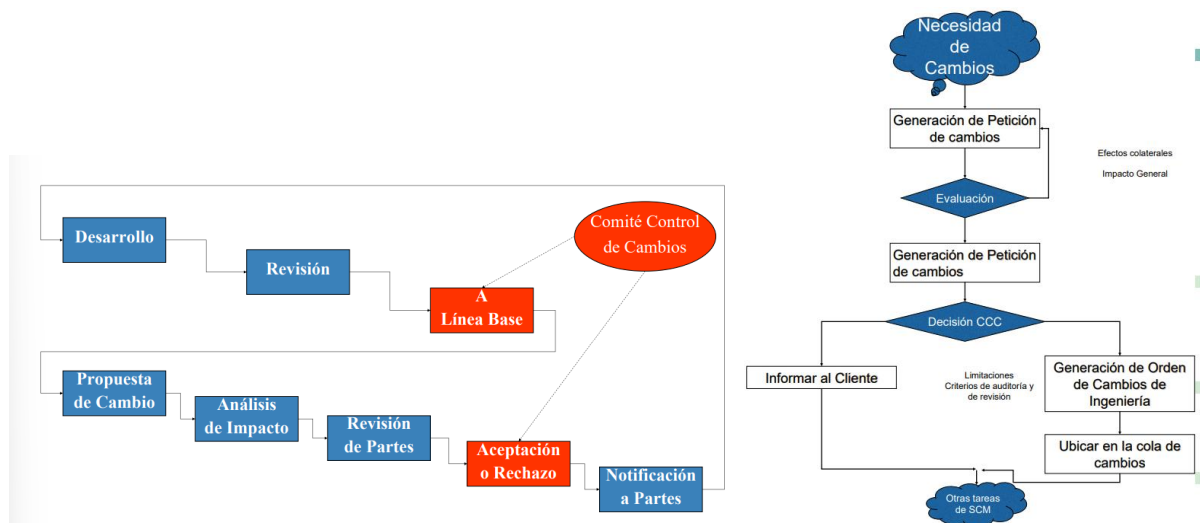
Ejemplos de IC



Control de Cambios

Tiene su origen en un requerimiento de cambio a uno o varios ítems de configuración que se encuentran en una línea base. Es un procedimiento formal que involucra diferentes actores y una evaluación del impacto del cambio.

Proceso de control de cambios → permite modificar el SW de manera controlada, manteniendo su integridad y trazabilidad.



1. **Desarrollo y revisión:** se elaboran los ítems de configuración y se revisa que cumplan con los criterios establecidos.
2. **Establecimiento de la línea base:** los ítems revisados y aprobados integran una configuración de referencia. A partir de ese momento, sus modificaciones requieren un procedimiento formal de control de cambios.
3. **Propuesta de cambio:** se registra qué se quiere modificar y por qué.
4. **Análisis de impacto:** se evalúan los efectos sobre otros ítems, los requisitos, las pruebas, los costos y los plazos.
5. **Revisión de partes:** las áreas o personas involucradas revisan la propuesta y sus consecuencias.
6. **Aceptación o rechazo:** el **Comité de Control de Cambios (CCC)** evalúa la propuesta y decide si la autoriza.
7. **Notificación a las partes:** se comunica la decisión. Si se aprueba, el cambio vuelve a desarrollo para implementarse, revisarse e incorporarse de forma controlada a la línea base. Si se rechaza, queda registrado y no se implementa.

El Comité de Control de Cambios → esta formado por representantes de todas las áreas involucradas en el desarrollo: análisis, diseño, implementación, testing, otros interesados.

Auditorías de Configuración de Software → verifican la coherencia entre: lo que se pidió, lo que realmente se construyó, y lo que quedó documentado. Es un proceso de evaluación externo objetivo. Se planifican por momentos, y se hacen sobre una línea base. Hay dos tipos de auditorías:

Auditoría Física de Configuración (PCA) → asegura que lo que se está indicando para cada ICs en la línea base o en la actualización se ha alcanzado realmente. Que lo que dice el plan este donde y cuando tiene que

estar. Se hace primero, porque antes de validar que se cumplan los requerimientos, etcétera, hay que ver si el repositorio está bien estructurado. Verifica que el producto construido sea consistente con su documentación.

Auditoría Funcional de Configuración (FCA) → evaluación independiente de los productos de SW, controlando que la funcionalidad y performance reales de cada CI sean consistentes con la especificación de requerimientos. Que lo que este, sea lo que tiene que estar. ¿El software hace realmente lo que debía hacer?

La **matriz de rastreabilidad de requerimientos** conecta los tres elementos: requerimientos, producto y documentación. Permite seguir cada requerimiento y saber: donde esta implementado, y como está documentado y probado.

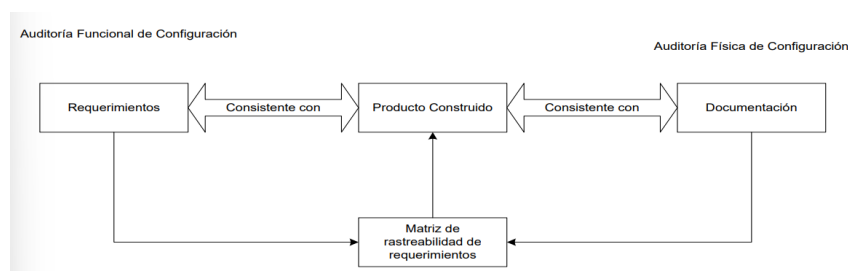
Así se puede verificar que:

- ningún requerimiento haya quedado sin implementar
- ninguna funcionalidad aparezca sin estar justificada por un requerimiento
- la documentación refleje realmente lo construido
- exista trazabilidad entre lo solicitado y el producto final.

La idea global del esquema es: para que una configuración sea íntegra, el producto debe cumplir los requerimientos y, al mismo tiempo, coincidir con la documentación que describe esa configuración. La matriz de trazabilidad ayuda a demostrar esa correspondencia.

Los auditores usan las **CHECKLIST** para hacer las auditorías → conjunto de preguntas a responder por lo auditado. Se debe hacer un seguimiento a los hallazgos hechos. Les sirven a los auditores como una guía estructurada para revisar la configuración y asegurarse de no olvidar aspectos importantes. El auditor toma una serie de criterios o preguntas previamente definidas y, para cada una, busca evidencia objetiva que demuestre si se cumple o no.

Sirven para que la auditoría sea sistemática y repetible: que distintos auditores revisen los mismos aspectos y que no dependa solamente de lo que a cada uno “se le ocurra mirar”.



Las auditorías de configuración sirven a dos procesos básicos:

Validación → si el problema es resuelto de manera apropiada y resuelve realmente la necesidad del usuario.

Verificación → Se comprueba que el producto cumpla con lo que fue especificado y establecido en las líneas base: requerimientos, diseño, documentación, casos de prueba, etc.

- La FCA comprueba que la funcionalidad y performance reales sean consistentes con los requerimientos.
- La PCA comprueba que lo que realmente existe en el producto coincida con lo declarado para los IC y la línea base.

Las auditorías aportan evidencia objetiva sobre la configuración para poder realizar ambas evaluaciones.

Informes de Estado

- Se ocupa de mantener los registros de la evolución del sistema.
- Maneja mucha información y salidas, por lo que se suele implementar dentro de procesos automáticos.
- Incluye reportes de estabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida. Permite tener una trazabilidad.

Plan de Gestión de Configuración

Define cómo se va a aplicar la Gestión de Configuración de Software en un proyecto: qué se va a controlar, quiénes serán responsables, qué procedimientos se seguirán y qué herramientas se utilizarán. Su función principal es establecer de antemano las reglas para identificar, controlar, auditar y hacer seguimiento de los ítems de configuración y sus cambios, evitando que SCM se realice de manera improvisada.

¿Qué debería incluir?

- Reglas de nombrado de los CI
- Herramientas a utilizar para SCM
- Roles e integrantes del Comité
- Procedimiento formal de cambios
- Plantillas de formularios
- Procesos de auditoría

Gestión de Configuración en ambientes ágiles

En un ambiente ágil, la Gestión de Configuración no desaparece, sino que cambia su enfoque: deja de verse como un mecanismo burocrático para “frenar” cambios y pasa a ser una forma de facilitar el trabajo del equipo, integrar cambios rápidamente y mantener la estabilidad del producto.

- Sirve a los practicantes (equipos de desarrollo) y no viceversa → las prácticas y herramientas de configuración deben adaptarse a la forma de trabajo del equipo, no obligar al equipo a trabajar de una manera incómoda solo por la herramienta.
- Coordina el trabajo, no controla a las personas → su objetivo es mantener sincronizados los cambios de todos, evitar conflictos y asegurar que el producto siga siendo integrable, no vigilar individualmente a los desarrolladores.
- Responde a los cambios en lugar de tratar de evitarlos.
- Esforzarse por ser transparente y “sin fricción”, automatizando tanto como sea posible.
- Coordinación y automatización frecuente y rápida.
- Eliminar el desperdicio – no agregar nada más que valor.
- Documentación Lean y Trazabilidad → mantener solo la documentación necesaria y útil, evitando duplicaciones.
- Feedback continuo y visible sobre calidad, estabilidad e integridad.

Algunos tips

- Es responsabilidad de todo el equipo.
- Automatizar lo más posible.
- Educar al equipo.
- Tareas de SCM embebidas en las demás tareas requeridas para alcanzar el objetivo del Sprint.

¿Qué pasa con el Comité de Control de Cambios? → el control de cambios sigue existiendo, pero se busca evitar un comité formal y burocrático para cada modificación. La responsabilidad se acerca al equipo y a los interesados correspondientes, de modo que los cambios puedan evaluarse e incorporarse rápidamente dentro de las iteraciones.

¿Qué ítems de configuración podemos tener? → Siguen existiendo ítems de configuración, porque todavía necesitamos identificar y controlar aquello necesario para construir y reproducir el producto. Lo que cambia es que se busca controlar solo aquello que aporte valor y sea necesario, evitando documentación innecesaria.

¿Qué pasa con las auditorías? → tampoco desaparecen, pero pueden realizarse de manera más frecuente y automatizada, integradas al proceso de desarrollo.

¿Qué pasa con los reportes de estado? → La información de estado sigue siendo necesaria, porque SCM debe permitir conocer qué versión existe, qué cambios se realizaron y cuál es el estado del producto. Lo que se busca en ágil es generarla automáticamente y hacerla visible de manera continua, en vez de producir grandes informes manuales.